

minigame/src/main.ts:1 至 minigame/src/main.ts:58

```
1  /**
2   * Mini-game entry point: boots the adapter, sets up the main canvas,
3   * routes touch events into the UI framework, and runs the main loop.
4   */
5   import { initAdapter, loadPixelFont, getMainCanvas } from './adapter';
6   import { Router, type TouchEvent, type TouchPoint } from './ui/framework';
7   import { lobbyState } from './lobby-state';
8   import { LobbyScreen } from './ui/lobby';
9   import { DeckBuilderScreen } from './ui/deck-builder';
10  import { ShopScreen } from './ui/shop';
11  import { BattleScreen } from './ui/battle';
12  import { preloadAllCardArt } from './ui/card-render';
13  declare const tt: any;
14  /**
15   * CDN base URL for art assets.
16   *
17   * 美术资源 ( 约 26MB WebP ) 不上传主包，运行时从此 CDN 地址下载。
18   * 上线前把 minigame-cdn/ 目录上传到你的 CDN，然后把下面的地址改成
19   * 你的 CDN 地址 ( 以 / 结尾 )。
20   *
21   * 留空则从本地包内加载 ( 仅开发调试用，需要把 art 目录也放进包 )。
22   */
23  const ART_CDN_BASE = 'https://cdn.example.com/pixel-frontline/';
24  /**
25   * BGM 等大音频文件的 CDN 地址。与美术资源同一个 CDN 即可，
26   * 把 minigame-cdn/audio/ 目录上传到 CDN 后自动生效。
27   */
28  const AUDIO_CDN_BASE = ART_CDN_BASE;
29  function toPoints(list: any): TouchPoint[] {
30    if (!list) return [];
31    const out: TouchPoint[] = [];
32    for (const t of list) {
33      out.push({
34        id: t.identifier ?? 0,
35        x: t.clientX,
36        y: t.clientY,
37      });
38    }
39    return out;
40  }
41  export function boot(): void {
42    initAdapter();
43    // Route art loads to the CDN (WebP versions).
44    if (ART_CDN_BASE) {
45      (globalThis as { __ART_CDN_BASE__: string }).__ART_CDN_BASE__ = ART_CDN_BASE;
46    }
47    if (AUDIO_CDN_BASE) {
48      (globalThis as { __AUDIO_CDN_BASE__: string }).__AUDIO_CDN_BASE__ = AUDIO_CDN_BASE;
49    }
50    const fontFamily = loadPixelFont();
```

minigame/src/main.ts:60 至 minigame/src/main.ts:117

```
1  const canvas = getMainCanvas();
2  const ctx = canvas.getContext('2d') as CanvasRenderingContext2D;
3  const info = tt.getSystemInfoSync();
4  const screenW = info.windowWidth;
5  const screenH = info.windowHeight;
6  const pixelRatio = info.pixelRatio || 1;
7  ctx.scale(pixelRatio, pixelRatio);
8  // — Router + screens —
9  const router = new Router();
10 const lobby = new LobbyScreen(screenW, screenH, router);
11 const builder = new DeckBuilderScreen(screenW, screenH, router);
12 const shop = new ShopScreen(screenW, screenH, router);
13 const battle = new BattleScreen(screenW, screenH, router);
14 router.register('lobby', lobby);
15 router.register('deck-builder', builder);
16 router.register('shop', shop);
17 router.register('battle', battle);
18 lobbyState.onBattle = () => {
19   battle.configure(lobbyState.match!);
20   router.navigate('battle');
21 };
22 router.navigate('lobby');
23 // — Touch routing —
24 const activeTouches = new Map<number, TouchPoint>();
25 const dispatch = (type: TouchEvent['type'], raw: any) => {
26   const changed = toPoints(raw.changedTouches ?? raw.touches);
27   if (type === 'down') {
28     for (const p of changed) activeTouches.set(p.id, p);
29   }
30   const points = [...activeTouches.values()];
31   router.handleTouch({ type, points, changed });
32   if (type === 'up') {
33     for (const p of changed) activeTouches.delete(p.id);
34   }
35 };
36 tt.onTouchStart((e: any) => dispatch('down', e));
37 tt.onTouchMove((e: any) => dispatch('move', e));
38 tt.onTouchEnd((e: any) => dispatch('up', e));
39 tt.onTouchCancel((e: any) => dispatch('up', e));
40 // — Main loop —
41 let last = Date.now();
42 const loop = () => {
43   const now = Date.now();
44   const dt = Math.min(0.1, (now - last) / 1000);
45   last = now;
46   router.update(dt);
47   ctx.clearRect(0, 0, screenW, screenH);
48   router.draw(ctx);
49   requestAnimationFrame(loop);
50 };
```

minigame/src/main.ts:118 至 game/acoustics.ts:26

```
1   requestAnimationFrame(loop);
2   // — Background preload of card art —
3   preloadAllCardArt();
4   // — Lifecycle: pause on hide —
5   if (typeof tt.onHide === 'function') {
6     tt.onHide() => {
7       battle.onAppHide();
8     };
9   }
10  if (typeof tt.onShow === 'function') {
11    tt.onShow() => {
12      battle.onAppShow();
13    };
14  }
15  if (typeof tt.onAudioInterruptionBegin === 'function') {
16    tt.onAudioInterruptionBegin() => {
17      battle.onAudioInterrupt(true);
18    };
19  }
20  if (typeof tt.onAudioInterruptionEnd === 'function') {
21    tt.onAudioInterruptionEnd() => {
22      battle.onAudioInterrupt(false);
23    };
24  }
25  // Keep a reference so the font family is not tree-shaken away.
26  void fontFamily;
27  }
28  boot();
29  /**
30   * Speed-of-sound model for battlefield audio.
31   *
32   * In a real firefight the muzzle flash arrives before the shot, and the
33   * brain reads that gap as distance. Every positional sound goes through
34   * `soundDelay` so distant booms lag behind their visuals, which is what
35   * makes a wide front *feel* wide.
36   */
37  /** Pixels per second (1 px ≈ 0.13 m, so this is roughly 340 m/s). */
38  export const SOUND_SPEED = 2600;
39  /** Inside this radius (px) the sound is effectively instantaneous. */
40  export const NEAR_FIELD = 220;
41  /** Longest lag (s) we allow — beyond this it stops sounding like distance. */
42  export const MAX_DELAY = 0.8;
43  /** Horizontal distance from a sound source to the listener (screen centre). */
44  export function listenerDistance(
45    x: number,
46    camera: number,
47    width: number,
48  ): number {
49    return Math.abs(x - (camera + width / 2));
50  }
```

game/acoustics.ts:28 至 game/adult-animation.ts:43

```

1  /** Travel time (s) for a sound at `distance` px, clamped to the near field. */
2  export function soundDelay(distance: number): number {
3      return Math.min(
4          Math.max((distance - NEAR_FIELD) / SOUND_SPEED, 0),
5          MAX_DELAY,
6      );
7  }
8  import { CARDS, type CardId } from './cards';
9  import type { Unit } from './engine';
10 import { grenadeCel, stanceTransitionActive, stanceTransitionProgress, stanceHeightClass, magazineReloadActive, magazineReloadCel } from './infantry-action-timing';
11 import { ammunition } from './ballistics';
12 import { crouchTravelAmount } from './crouch-locomotion';
13 import { proneTravelAmount } from './prone-locomotion';
14 import { supportWorkSettled } from './support-work';
15 export type AdultIdentity = 'infantry' | 'marines' | 'police' | 'militia';
16 export interface AdultSprites {
17     walk8: HTMLCanvasElement[];
18     crouch8: HTMLCanvasElement[];
19     actions20: HTMLCanvasElement[];
20     /** Original matched low-crawl pair, independent of the new settled poses. */
21     crawl2: HTMLCanvasElement[];
22     reactions8: HTMLCanvasElement[];
23     /** v121: dedicated hand-signal frames — point fwd, wave overhead, point back, fist. */
24     signals4: HTMLCanvasElement[];
25     reload8: HTMLCanvasElement[];
26     grenade8: HTMLCanvasElement[];
27     /** Painted stand→knee→prone cels, played backwards when rising. */
28     stance16: HTMLCanvasElement[];
29     /** Eight fixed-knee and eight prone magazine drills. */
30     lowReload16: HTMLCanvasElement[];
31     /** Eight medical-work cels at each of stand, knee and prone height. */
32     medical24: HTMLCanvasElement[];
33     /** Sixteen fixed-knee and sixteen prone throwing cels. */
34     lowGrenade32: HTMLCanvasElement[];
35     /** Twelve standing/knee and ten prone tool-work cels. */
36     repair34: HTMLCanvasElement[];
37     /** Subtle whole-body prone observation; never a raised crawling silhouette. */
38     proneIdle8: HTMLCanvasElement[];
39 }
40 export interface AdultFrameChoice {
41     group: keyof AdultSprites;
42     index: number;
43     /**
44      * Visual facing override for the renderer's sprite flip. Sector scans set
45      * this so the soldier visibly glances toward a flank without touching the
46      * unit's real facing, which keeps driving muzzle direction and movement.
47      */
48     dir?: 1 | -1;
49 }
50 /** These are whole-body authored actions, not bases for a decorative torso. */

```

game/adult-animation.ts:44 至 game/adult-animation.ts:94

```

1  export function ownsAdultBody(choice: AdultFrameChoice | null): boolean {
2      return !!choice && ['stance16','reload8','lowReload16','grenade8','lowGrenade32','reactions8','medical24','repair34','proneIdle8'].includes(choice.group);
3  }
4  export function adultIdentity(id: CardId): AdultIdentity {
5      if (id === 'militia') return 'militia';
6      if (CARDS[id].uniform === 'police') return 'police';
7      if (
8          CARDS[id].uniform === 'marine' ||
9          ['marines', 'paratroopers', 'rangers'].includes(id)
10     )
11         return 'marines';
12     return 'infantry';
13 }
14 const action = (index: number): AdultFrameChoice => ({
15     group: 'actions20',
16     index,
17 });
18 const reaction = (index: number): AdultFrameChoice => ({
19     group: 'reactions8',
20     index,
21 });
22 const sig = (index: number): AdultFrameChoice => ({
23     group: 'signals4',
24     index,
25 });
26 const cycle = (walk: number, length: number) =>
27     ((Math.floor(walk) % length) + length) % length;
28 /**
29  * Idle micro-motion: a soldier holding position briefly shifts to an alert
30  * stance or takes a knee to scan, so the front line never freezes into
31  * statues. Returns null most of the time; the caller falls back to the
32  * normal idle frame. Phase is offset by uid so squads don't move in sync.
33  */
34 export function idleMicroChoice(u: Unit, time: number): AdultFrameChoice | null {
35     if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
36     if (u.suppression > 0.4) return null;
37     if (u.digging || u.tending || u.draggingUid !== undefined) return null;
38     // v118: standing-only. A crouched or prone soldier who popped back to the
39     // standing alert frame every few seconds read as a jack-in-the-box; the
40     // low poses have their own fidget layers below.
41     if (u.pose !== 'idle') return null;
42     // Command vacuum: leaderless soldiers glance around nervously, cycling
43     // between alert stance and a knee-scan on a short, irregular cadence so
44     // the disorganisation reads visually without any UI hint.
45     if (u.vacuum) {
46         const phase = (time + u.uid * 3.91) % 4.6;
47         if (phase < 1.1) return action(0);
48         if (phase < 2.0) return action(0);
49         return null;
50     }

```

game/adult-animation.ts:95 至 game/adult-animation.ts:146

```

1  // Idle vigilance can turn the soldier, but cannot change body height.
2  // All actual kneeling/standing decisions belong to the stance gate.
3  const period = 9 + (u.uid % 5) * 1.3;
4  const phase = (time + u.uid * 7.31) % period;
5  if (phase < 3.3) return action(0);
6  if (phase < 3.9) {
7    // rise and glance over the shoulder toward the covered flank
8    const front = (u.facing < 0 ? -1 : 1) as 1 | -1;
9    return { ...action(0), dir: (front * -1) as 1 | -1 };
10 }
11 return null;
12 }
13 /**
14  * v118: crouch idle fidget — a soldier holding a knee periodically shifts
15  * into a low crouch or leans forward to work the sector, so a dug-in line
16  * doesn't freeze into identical statues. Returns null most of the time; the
17  * caller falls back to the static kneel. Animation-only, like the other
18  * idle layers. The period drifts with uid so neighbours desynchronise.
19  */
20 export function crouchFidgetChoice(
21   u: Unit,
22   time: number,
23 ): AdultFrameChoice | null {
24   if (u.pose !== 'crouch') return null;
25   if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
26   if (u.suppression > 0.4) return null;
27   if (u.digging || u.tending || u.draggingUid !== undefined) return null;
28   if (u.vacuum) return null;
29   if ((u.fragThrow ?? 0) > 0) return null;
30   const period = 10 + (u.uid % 4) * 0.9;
31   const phase = (time + u.uid * 6.13) % period;
32   if (phase < 1.4) return action(1); // action 17 is a running frame, not a crouch
33   if (phase < 2.4) return action(1); // 13 is crawling on hands/knees, not a knee scan
34   return null;
35 }
36 /**
37  * Quiet observation stays at the committed prone height. The legacy
38  * radio/crawl cels raise the torso by 6–8px and cannot be idle substitutes.
39  */
40 export function proneFidgetChoice(
41   u: Unit,
42   time: number,
43 ): AdultFrameChoice | null {
44   if (u.pose !== 'prone') return null;
45   if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
46   if (u.suppression > 0.4) return null;
47   if (u.digging || u.tending || u.draggingUid !== undefined) return null;
48   if (u.vacuum) return null;
49   if ((u.fragThrow ?? 0) > 0) return null;
50   return {group:'proneIdle8',index:Math.floor((time+u.uid*.73)/.6)%8};

```

game/adult-animation.ts:147 至 game/adult-animation.ts:198

```

1  }
2  /**
3   * Sector scan: a soldier holding a position works through a structured scan
4   * cycle — alert check to the front, take a knee and glance over the shoulder
5   * toward the covered flank, then rise back to the alert stance. The scan is
6   * animation-only: the returned `dir` overrides the sprite flip in the
7   * renderer while the unit's real facing (muzzle direction, movement) is
8   * untouched, and the omni-directional sight system means the scan never
9   * changes what the soldier can actually see. Pure O(1) arithmetic; the phase
10  * is offset by uid so a squad's scans interleave instead of rippling in
11  * sync, and the period stretches slightly with uid so neighbours drift out
12  * of phase over time.
13  */
14  export function sectorScanChoice(
15    u: Unit,
16    time: number,
17  ): AdultFrameChoice | null {
18    if (u.hp <= 0 || u.wounded || u.surrendered) return null;
19    if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
20    if (u.suppression > 0.4) return null;
21    if (u.digging || u.tending || u.draggingUid !== undefined) return null;
22    if (u.vacuum) return null;
23    if ((u.fragThrow ?? 0) > 0) return null;
24    // Scanning is a deliberate hold behaviour: only from a standing idle pose.
25    // Crouch/prone/hunker poses have their own glance cycles.
26    if (u.pose !== 'idle') return null;
27    // 23 s + up to ~8.5 s, so a six-man squad never locks into a shared rhythm.
28    const period = 23 + (u.uid % 6) * 1.7;
29    const phase = (time + u.uid * 5.77) % period;
30    if (phase >= 3.0) return null;
31    const front = (u.facing < 0 ? -1 : 1) as 1 | -1;
32    const rear = (front * -1) as 1 | -1;
33    // Beat 1 (0.0–0.9 s): alert stance, eyes front.
34    if (phase < 0.9) return { ...action(0), dir: front };
35    // Beat 2 (0.9–2.0 s): check the flank without taking an unscheduled knee.
36    if (phase < 2.0) return { ...action(0), dir: rear };
37    // Beat 3 (2.0–3.0 s): rise back to the alert stance, eyes front.
38    return { ...action(0), dir: front };
39  }
40  /**
41   * Blast glance: a distant explosion snags a holding soldier's attention —
42   * they snap to the alert stance and look toward the impact for a beat. Like
43   * the sector scan this is animation-only: the returned `dir` flips the sprite
44   * while the unit's real facing (muzzle direction, movement) is untouched, and
45   * the omni-directional sight system means the glance never changes what the
46   * soldier can see. Takes priority over the routine sector scan because a
47   * fresh blast is far more salient than a scheduled sweep.
48   */
49  export function blastGlanceChoice(
50    u: Unit,

```

game/adult-animation.ts:199 至 game/adult-animation.ts:250

```

1   time: number,
2   ): AdultFrameChoice | null {
3     if ((u.blastGlanceUntil ?? 0) <= time) return null;
4     if (u.hp <= 0 || u.wounded || u.surrendered) return null;
5     if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
6     if (u.suppression > 0.4) return null;
7     if (u.digging || u.tending || u.draggingUid !== undefined) return null;
8     if (u.vacuum) return null;
9     if ((u.fragThrow ?? 0) > 0) return null;
10    if (u.pose !== 'idle') return null;
11    return { ...action(0), dir: u.blastGlanceDir ?? 1 };
12  }
13  /**
14   * Dug-in blast glance: the same distant-blast awareness as
15   * {@link blastGlanceChoice}, but for soldiers holding a low crouch or prone
16   * position. They keep their profile down — a crouching defender stays on one
17   * knee, a prone defender stays on the deck — and only turn to track the
18   * impact. Animation-only, exactly like the standing glance: the returned
19   * `dir` flips the sprite while the real facing and the omni-directional
20   * sight system are untouched. Heavily suppressed soldiers (hunker pose) are
21   * gated out by the suppression check, so a pinned defender keeps their head
22   * down instead of looking up at the shellburst.
23   */
24  export function dugInBlastGlanceChoice(
25    u: Unit,
26    time: number,
27  ): AdultFrameChoice | null {
28    if ((u.blastGlanceUntil ?? 0) <= time) return null;
29    if (u.hp <= 0 || u.wounded || u.surrendered) return null;
30    if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
31    if (u.suppression > 0.4) return null;
32    if (u.digging || u.tending || u.draggingUid !== undefined) return null;
33    if (u.vacuum) return null;
34    if ((u.fragThrow ?? 0) > 0) return null;
35    if (u.pose === 'crouch') return { ...action(1), dir: u.blastGlanceDir ?? 1 };
36    // A head glance is not an instantaneous 180° turn of a prone body/rifle.
37    if (u.pose === 'prone') return action(2);
38    return null;
39  }
40  /**
41   * Trace glance: fresh drag marks on the ground — blood left by a casualty
42   * hauled across the dirt — snag a holding soldier's attention for a beat.
43   * Animation-only, exactly like the blast glance: the returned `dir` flips
44   * the sprite toward the trace while the unit's real facing and the
45   * omni-directional sight system are untouched. Ranks below the blast glance
46   * because an explosion is more salient than a blood smear, but above the
47   * routine sector scan because a fresh trail means recent contact nearby.
48   */
49  export function traceGlanceChoice(
50    u: Unit,

```

game/adult-animation.ts:251 至 game/adult-animation.ts:302

```

1   time: number,
2   ): AdultFrameChoice | null {
3     if ((u.traceGlanceUntil ?? 0) <= time) return null;
4     if (u.hp <= 0 || u.wounded || u.surrendered) return null;
5     if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
6     if (u.suppression > 0.4) return null;
7     if (u.digging || u.tending || u.draggingUid !== undefined) return null;
8     if (u.vacuum) return null;
9     if ((u.fragThrow ?? 0) > 0) return null;
10    if (u.pose !== 'idle') return null;
11    return { ...action(0), dir: u.traceGlanceDir ?? 1 };
12  }
13  /**
14   * Dug-in trace glance: the same blood-trail awareness as
15   * {@link traceGlanceChoice}, but for soldiers holding a low crouch or prone
16   * position. They keep their profile down and only turn to track the trace.
17   * Animation-only, with the same gating as the dug-in blast glance — pinned
18   * defenders keep their heads down instead of looking at the blood.
19   */
20  export function dugInTraceGlanceChoice(
21    u: Unit,
22    time: number,
23  ): AdultFrameChoice | null {
24    if ((u.traceGlanceUntil ?? 0) <= time) return null;
25    if (u.hp <= 0 || u.wounded || u.surrendered) return null;
26    if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
27    if (u.suppression > 0.4) return null;
28    if (u.digging || u.tending || u.draggingUid !== undefined) return null;
29    if (u.vacuum) return null;
30    if ((u.fragThrow ?? 0) > 0) return null;
31    if (u.pose === 'crouch')
32      return { ...action(1), dir: u.traceGlanceDir ?? 1 };
33    if (u.pose === 'prone') return action(2);
34    return null;
35  }
36  /**
37   * Composed idle pose for the renderer: a structured sector scan takes
38   * precedence over the random idle micro-motion, so the two never fight over
39   * the same frame. A fresh blast glance outranks both — an explosion always
40   * interrupts a routine scan. Dug-in defenders (crouch/prone) get their own
41   * glance layer between the standing glance and the routine scan. A fresh
42   * blood-trail glance ranks just below the blast glance — the trail means
43   * recent contact, but a shellburst is louder. Returns null when the soldier
44   * should hold the default patrol idle frame.
45   */
46  export function idlePoseChoice(u: Unit, time: number): AdultFrameChoice | null {
47    // Do not let decorative layers replace a throw, reload, or an authored
48    // stance transition. They must never invent a different body height.
49    if (u.hp <= 0 || u.wounded || u.surrendered || u.rappelling || u.parachuting ||
50        u.climbing > 0 || u.motion !== 'ground' || u.secondaryFire > 0 ||

```

game/adult-animation.ts:303 至 game/adult-animation.ts:354

```

1      (u.fragThrow ?? 0) > 0 || (u.reloadUntil ?? 0) > time ||
2      stanceTransitionActive(u, time) || u.flash > 0 || u.tending ||
3      (u.firstAidUntil ?? 0) > time) return null;
4  return (
5      heardContactGlanceChoice(u, time) ??
6      blastGlanceChoice(u, time) ??
7      dugInBlastGlanceChoice(u, time) ??
8      traceGlanceChoice(u, time) ??
9      dugInTraceGlanceChoice(u, time) ??
10     boundingRestChoice(u, time) ??
11     crouchFidgetChoice(u, time) ??
12     proneFidgetChoice(u, time) ??
13     // Legacy decorative checks reused a hands-and-knees crawl. Only actual
14     // reload/treatment actions may use their work cels; idle cannot fake one.
15     sectorScanChoice(u, time) ??
16     idleMicroChoice(u, time)
17 );
18 }
19 /**
20  * Contact callout: the beat after a soldier spots the enemy, they shout the
21  * contact to their squad — arm up and pointing at the threat, alternating
22  * with the alert stand so the shout reads as a wave, not a statue.
23  * Animation-only, exactly like the glance layers: the returned `dir` points
24  * the shout at the contact without touching the unit's real facing. Only
25  * upright, unengaged soldiers call out — a man already firing, aiming or
26  * reloading keeps the weapon on the threat instead of waving.
27  */
28 export function contactCalloutChoice(
29     u: Unit,
30     time: number,
31 ): AdultFrameChoice | null {
32     if ((u.calloutUntil ?? 0) <= time) return null;
33     if (u.moving || u.fire > 0) return null;
34     if ((u.aimUntil ?? 0) > time) return null;
35     if ((u.reloadUntil ?? 0) > time) return null;
36     if (u.pose !== 'idle' && u.pose !== 'walk') return null;
37     const dir = u.calloutDir ?? 1;
38     const wave = Math.floor(((u.calloutUntil ?? 0) - time) * 6) % 2;
39     // v128: dedicated hand-signal frames — the old action(9) is the climb
40     // pose (arm up + knee raised), so a contact shout read as a man scaling
41     // a wall in the middle of a field. sig(1) waves overhead, sig(0) points.
42     return wave ? { ...sig(1), dir } : { ...sig(0), dir };
43 }
44 /**
45  * v118: leader point-out — while in contact, a squad leader periodically
46  * points an arm at the threat so the squad orients on the right target.
47  * Animation-only, exactly like the callout: the returned `dir` points the
48  * gesture without touching the unit's real facing. Only upright, unengaged
49  * leaders point — a man already firing, aiming or reloading keeps the
50  * weapon on the threat instead of waving.

```

game/adult-animation.ts:355 至 game/adult-animation.ts:406

```

1  */
2  export function leaderPointChoice(
3      u: Unit,
4      time: number,
5  ): AdultFrameChoice | null {
6      if ((u.pointUntil ?? 0) <= time) return null;
7      if (u.moving || u.fire > 0) return null;
8      if ((u.aimUntil ?? 0) > time) return null;
9      if (u.suppression > 0.4) return null;
10     if (u.pose !== 'idle' && u.pose !== 'walk') return null;
11     if (u.digging || u.tending || u.draggingUid !== undefined) return null;
12     if ((u.fragThrow ?? 0) > 0) return null;
13     // v128: sig(0) point-forward instead of the climb pose (see above).
14     return { ...sig(0), dir: u.pointDir ?? 1 };
15 }
16 /**
17  * Heard-contact glance: a squad mate's shout travels faster than the threat.
18  * For a beat after `heardContactAt`, a soldier who hasn't spotted the enemy
19  * themselves snaps to the alert stand and looks toward the shouted bearing,
20  * so a platoon reacts as one when contact is called. Animation-only: the
21  * `dir` flip never changes the unit's real facing or what the omni-sight
22  * system can see. Suppressed once the soldier has made their own contact —
23  * a man in the fight has no attention left for second-hand shouts.
24  */
25 export function heardContactGlanceChoice(
26     u: Unit,
27     time: number,
28 ): AdultFrameChoice | null {
29     if (u.heardContactAt === undefined) return null;
30     if (time - u.heardContactAt >= 0.7) return null;
31     if (u.moving || u.fire > 0) return null;
32     if ((u.reloadUntil ?? 0) > time) return null;
33     if (u.pose !== 'idle') return null;
34     if ((u.contactUntil ?? 0) > time) return null;
35     return { ...action(0), dir: u.heardContactDir ?? 1 };
36 }
37 /**
38  * Bounding rest: after a fireteam finishes a bound, the lead element drops to
39  * a knee for a beat to catch its breath while the overwatch element moves.
40  * The engine sets `boundRestUntil` at the end of a bound; the soldier holds
41  * the single-knee frame until it expires, so the fireteam-manoeuve rhythm
42  * reads on the field instead of every man standing tall the whole time.
43  */
44 export function boundingRestChoice(
45     u: Unit,
46     time: number,
47 ): AdultFrameChoice | null {
48     if ((u.boundRestUntil ?? 0) <= time) return null;
49     if (u.moving || u.fire > 0) return null;
50     if ((u.reloadUntil ?? 0) > time) return null;

```

game/adult-animation.ts:407 至 game/adult-animation.ts:458

```
1   if (u.pose !== 'idle') return null;
2   return action(0);
3 }
4 /**
5  * v120: magazine check — a rifleman holding position periodically hunches
6  * over the mag well and seats/checks the magazine, so a held line reads as
7  * professionals maintaining their kit instead of statues. Animation-only.
8  * Only for units with magazine-managed ammo (ammo >= 0); crews and energy
9  * weapons have no mag to check.
10  */
11 export function magCheckChoice(
12   u: Unit,
13   time: number,
14 ): AdultFrameChoice | null {
15   if (u.ammo === undefined || u.ammo < 0) return null;
16   if (u.hp <= 0 || u.wounded || u.surrendered) return null;
17   if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
18   if (u.suppression > 0.4) return null;
19   if (u.digging || u.tending || u.draggingUid !== undefined) return null;
20   if (u.vacuum) return null;
21   if ((u.fragThrow ?? 0) > 0) return null;
22   if (u.pose !== 'crouch') return null;
23   const period = 21 + (u.uid % 5) * 1.3;
24   const phase = (time + u.uid * 6.83) % period;
25   if (phase >= 2.6) return null;
26   return action(1); // no dedicated kit-check drill yet: retain the planted knee
27 }
28 /**
29  * v120: engineer fuss — a sapper holding position periodically drops to a
30  * knee and fusses with his kit (detonator, charges, tools), so the squad's
31  * technical specialist reads as a man with a job to do even when idle.
32  * Animation-only. Only for engineer-trait cards.
33  */
34 export function engineerFussChoice(
35   u: Unit,
36   time: number,
37 ): AdultFrameChoice | null {
38   if (CARDS[u.id].trait !== 'engineer') return null;
39   if (u.hp <= 0 || u.wounded || u.surrendered) return null;
40   if (u.moving || u.fire > 0 || (u.aimUntil ?? 0) > time) return null;
41   if (u.suppression > 0.4) return null;
42   if (u.digging || u.tending || u.draggingUid !== undefined) return null;
43   if (u.vacuum) return null;
44   if ((u.fragThrow ?? 0) > 0) return null;
45   if (u.pose !== 'crouch') return null;
46   const period = 16 + (u.uid % 4) * 1.2;
47   const phase = (time + u.uid * 5.29) % period;
48   if (phase >= 2.8) return null;
49   return action(1); // frame 13 is crawling, not kneeling kit work
50 }
```

game/adult-animation.ts:460 至 game/adult-animation.ts:512

```

1  /** Dedicated painted progression, using the simulation's posture clock.
2  * Reload/fire/decoration cannot replace these cels midway through a drill. */
3  const POSE_CHAINS: Record<
4    'stand' | 'crouch' | 'prone',
5    Partial<Record<'stand' | 'crouch' | 'prone', number[]>>
6  > = {
7    stand: { crouch: [0,1,2,3,4,5,6,7], prone: [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] },
8    crouch: { stand: [7,6,5,4,3,2,1,0], prone: [8,9,10,11,12,13,14,15] },
9    prone: { stand: [15,14,13,12,11,10,9,8,7,6,5,4,3,2,1,0], crouch: [15,14,13,12,11,10,9,8] },
10 };
11 /** Work may move the arms, but never invent a new stance behind the AI's back. */
12 function groundedWork(u: Unit, _time: number): AdultFrameChoice {
13   if (u.pose === 'prone') return action(2);
14   if (u.pose === 'crouch' || u.pose === 'hunker')
15     return action(1);
16   return action(0);
17 }
18 /** Only a real treatment task may own the painted medical body. */
19 export function medicalWorkChoice(u: Unit, time: number): AdultFrameChoice | null {
20   if (u.hp <= 0 || u.wounded || u.surrendered || u.rappelling || u.parachuting ||
21     u.moving || u.motion !== 'ground' || u.climbing > 0 ||
22     stanceTransitionActive(u, time) || crouchTravelAmount(u) > 0) return null;
23   const aid = (u.firstAidUntil ?? 0) > time;
24   const medical = u.tending && u.tendingKind === 'medical';
25   if (!aid && !medical) return null;
26   const pose = stanceHeightClass(u.pose);
27   const elapsed = aid ? Math.max(0, time - (u.firstAidUntil! - 1.5)) : u.tendingTime ?? 0;
28   const beat = aid ? Math.min(7, Math.floor(elapsed / 1.5 * 8))
29     : Math.floor(Math.max(0, elapsed) / .3) % 8;
30   return { group: 'medical24', index: (pose === 'stand' ? 0 : pose === 'crouch' ? 8 : 16) + beat };
31 }
32 export function repairWorkChoice(u: Unit, time: number): AdultFrameChoice | null {
33   if (!u.tending || u.tendingKind !== 'repair' || CARDS[u.id].trait !== 'mechanic' ||
34     !supportWorkSettled(u,time)) return null;
35   const height = stanceHeightClass(u.pose);
36   const offset = height === 'stand' ? 0 : height === 'crouch' ? 12 : 24;
37   const count = height === 'prone' ? 10 : 12;
38   return { group:'repair34', index:offset + Math.floor((Math.max(0,u.tendingTime ?? 0)+1e-9)/2.4*count)%count };
39 }
40 /** One actual magazine clock, with authored cels at the committed height. */
41 function reloadBeat(u: Unit, time: number): AdultFrameChoice {
42   // Belt/launcher work must keep its own weapon, not become a rifle drill.
43   // The heavy gun's dedicated belt cels are applied by its renderer.
44   if (ammunition(u.id, u.member) !== 'rifle')
45     return action(u.pose === 'prone' ? 2 : stanceHeightClass(u.pose) === 'crouch' ? 1 : 0);
46   const beat = magazineReloadCel(u,time);
47   if (u.pose === 'prone') return { group: 'lowReload16', index: 8 + beat };
48   if (stanceHeightClass(u.pose) === 'crouch') return { group: 'lowReload16', index: beat };
49   return { group: 'reload8', index: beat };
50 }

```

game/adult-animation.ts:514 至 game/adult-animation.ts:564

```

1  export function poseTransitionChoice(
2    u: Unit,
3    time: number,
4  ): AdultFrameChoice | null {
5    const progress = stanceTransitionProgress(u, time);
6    if (progress === null || !u.poseAnimFrom || !u.poseAnimSeen) return null;
7    let chain = POSE_CHAINS[u.poseAnimFrom]?.[u.poseAnimSeen];
8    if (!chain) return null;
9    if (u.poseAnimFrom === 'crouch' || u.poseAnimSeen === 'crouch') {
10     const low = 7-Math.round((u.poseAnimFrom === 'crouch' ? u.poseAnimFromTravel ?? 0 : u.poseAnimToTravel ?? 0)*5);
11     const start = u.poseAnimFrom === 'stand' ? 0 : u.poseAnimFrom === 'prone' ? 15 : low;
12     const end = u.poseAnimSeen === 'stand' ? 0 : u.poseAnimSeen === 'prone' ? 15 : low;
13     if (low < 7) chain = Array.from({length:Math.abs(end-start)+1},(_,i)=>start+i*Math.sign(end-start));
14   }
15   // Leaving a supported crawl starts at its real low torso, not the flat
16   // aim endpoint. Reuse only the authored prone tail, never a knee/climb cel.
17   if ((u.poseAnimFrom === 'prone' && (u.poseAnimFromTravel ?? 0)>0) ||
18     (u.poseAnimSeen === 'prone' && (u.poseAnimToTravel ?? 0)>0)) {
19     const start=u.poseAnimFrom==='prone' ? 15-Math.round((u.poseAnimFromTravel??0)*4) : chain[0];
20     const end=u.poseAnimSeen==='prone' ? 15-Math.round((u.poseAnimToTravel??0)*4) : chain[chain.length-1];
21     chain=Array.from({length:Math.abs(end-start)+1},(_,i)=>start+i*Math.sign(end-start));
22   }
23   return { group: 'stance16', index: chain[Math.min(chain.length - 1, Math.floor(progress * chain.length))] };
24 }
25 /** Every living, casualty and surrender state uses the same adult anatomy. */
26 export function adultFrameChoice(u: Unit, time = 0): AdultFrameChoice {
27   // Reverse the existing raised-rifle gait while the body keeps facing contact.
28   const step = u.backpedaling ? -Math.floor(u.walk) : u.walk;
29   if (u.hp <= 0) return action(15);
30   // v133: surrender is checked BEFORE the rope/parachute branch. A stuck
31   // rappelling flag (morale break mid-descent) must never override hands-up —
32   // that ordering was how whole squads froze in the swim-lane climb pose.
33   if (u.surrendered)
34     return reaction(
35       u.surrenderTime < 0.35
36       ? 0
37       : u.surrenderTime < 0.75
38       ? 1
39       : u.surrenderTime < 2.5
40       ? 2
41       : 3,
42     );
43   if (u.wounded) {
44     if (u.crawling)
45       return { group: 'crawl2', index: cycle(u.walk * 2, 2) };
46     if (u.woundedFromPose === 'prone' || u.woundedTime >= 0.7)
47       return action(14);
48     const low = u.woundedFromPose === 'crouch' || u.woundedFromPose === 'land';
49     return reaction(
50       (low ? 5 : 4) + Math.min(low ? 2 : 3, Math.floor(u.woundedTime * 6)),

```

game/adult-animation.ts:565 至 game/adult-animation.ts:614

```

1   );
2   }
3   // Rope poses are only for healthy soldiers actively descending.
4   if (u.rappelling) return action(8 + cycle(u.walk / 2, 2));
5   const poseTransition = poseTransitionChoice(u, time);
6   if (poseTransition && u.motion === 'ground') return poseTransition;
7   const proneTravel=proneTravelAmount(u);
8   if(proneTravel>0 && (u.motion==='ground'||u.motion==='bank') && u.climbing<=0) {
9     if(proneTravel<1)return {group:'stance16',index:15-Math.round(proneTravel*4)};
10    return {group:'crawl2',index:cycle(u.walk/2,2)};
11  }
12  const lowTravel = crouchTravelAmount(u);
13  if (lowTravel > 0 && u.motion === 'ground' && u.climbing <= 0) {
14    if (lowTravel < 1) return { group:'stance16', index:7-Math.round(lowTravel*5) };
15    // Brief traffic/aim stops freeze the last real footstep, not a lower body.
16    return { group:'crouch8', index:cycle(step,8) };
17  }
18  // Release at 5/8 of the shared drill: after five standing or ten low cels.
19  if ((u.fragThrow ?? 0) > 0) {
20    if (stanceHeightClass(u.pose) === 'stand')
21      return { group: 'grenade8', index: grenadeCel(u, time, 8) };
22    return { group: 'lowGrenade32', index: (u.pose === 'prone' ? 16 : 0) + grenadeCel(u, time, 16) };
23  }
24  const medical = medicalWorkChoice(u, time);
25  if (medical) return medical;
26  const repair = repairWorkChoice(u,time);
27  if (repair) return repair;
28  // Repair is a different job. It must not borrow bandaging or crawl cels.
29  if (u.tending && !u.moving && u.motion === 'ground') return groundedWork(u, time);
30  // While changing a cooked barrel the gunner drops to one knee and works the
31  // weapon, alternating with a low crouch so the pause reads as urgent labour.
32  if ((u.overheatedUntil ?? 0) > time && !u.moving)
33    return groundedWork(u, time);
34  if (u.motion === 'jump') return action(u.motionTime < 0.12 ? 4 : 5);
35  if (u.motion === 'land')
36    return action(u.motionTime < u.motionDuration * 0.5 ? 6 : 7);
37  // v130: walls were removed from the map generator (wallSites: []), so
38  // u.climbing is never set in normal play. The old branch returned the
39  // climb frames (8-11) which read as the swim-lane pose; if climbing ever
40  // comes back it must use a dedicated vault animation, never these frames.
41  // Until then, fall through to the crouch gait like a bank out of cover.
42  if (u.climbing > 0)
43    return { group: 'crouch8', index: cycle(Math.floor(u.motionTime * 8), 8) };
44  if (u.motion === 'bank')
45    return u.pose === 'prone' ? { group: 'crawl2', index: cycle(u.walk / 2, 2) }
46      : u.pose === 'crouch' || u.pose === 'hunker'
47      ? { group: 'crouch8', index: cycle(step, 8) }
48      : { group: 'walk8', index: cycle(step, 8) };
49  // A receiving soldier actually replaces the magazine. Sharing/observing
50  // must not hide the drill, nor may a decorative task restart its clock.

```

game/adult-animation.ts:615 至 game/adult-animation.ts:664

```

1  if (!u.moving && magazineReloadActive(u, time)) return reloadBeat(u, time);
2  // v81: dry-ammo battle drill. The engine sets reloadingUntil on the dry
3  // receiver only, so during the handoff the pair splits into a giver (arm
4  // extended with the magazine) and a receiver (hunched over the mag well)
5  // instead of two identical hunches. Only upright soldiers run the drill —
6  // a pinned rifleman stays low and waits for a lull in the fire.
7  if ((u.ammoShareUntil ?? 0) > time && u.pose !== 'prone')
8  // v128: giver extends an arm (sig point-forward) instead of the climb pose.
9  return u.pose === 'crouch' || u.pose === 'hunker' ? action(1) : action(0);
10 // v83: looting a fallen comrade's kit — a knee-down rummage beat
11 // alternating with the huddled work beat so the search reads as active.
12 if ((u.scavengeUntil ?? 0) > time && u.pose !== 'prone')
13 return groundedWork(u, time);
14 if (
15 (u.ammoSignalUntil ?? 0) > time &&
16 !u.moving &&
17 u.fire <= 0 &&
18 (u.aimUntil ?? 0) <= time &&
19 (u.reloadUntil ?? 0) <= time &&
20 (u.pose === 'idle' || u.pose === 'walk')
21 ) {
22 // The ammunition UI already communicates this state. No raised-arm
23 // command animation, and no decorative drop to a knee.
24 return groundedWork(u, time);
25 }
26 // v119: weapon crews work the gun while emplacing. MG / AT gun / mortar
27 // setup is 1.2s of labour — the crew alternates the bent-work beat with
28 // the kneel so they read as hauling the weapon into position instead of
29 // standing frozen beside it. Prone/hunker crews stay low (they're pinned,
30 // not emplacing), and moving crews are handled by the gait branches below.
31 if (
32 (u.emplacementSetupUntil ?? 0) > time &&
33 !u.moving &&
34 u.pose !== 'prone' &&
35 u.pose !== 'hunker'
36 )
37 return groundedWork(u, time);
38 // Stance transitions are resolved above the work actions, before arrival
39 // at the stable prone/crouch frames below.
40 if (u.pose === 'prone') {
41 if (u.moving) return { group: 'crawl2', index: cycle(u.walk / 2, 2) };
42 // Observation uses the same grounded body as ordinary prone infantry.
43 // The old periodic radio cel was 6–8px taller and bypassed stance timing.
44 // Keep the aimed torso planted; discharge effects carry weapon recoil.
45 if (u.fire > 0 || u.secondaryFire > 0) return action(2);
46 if ((u.aimUntil ?? 0) > time) return action(2);
47 return action(2);
48 }
49 if (u.pose === 'crouch') {
50 if (u.moving) {

```

game/adult-animation.ts:665 至 game/adult-animation.ts:714

```

1      // Hauling a casualty: a slow, heavy gait at half the cadence of a
2      // normal crouch-walk so the drag reads as effort, not patrol.
3      const gait = u.draggingUid !== undefined ? u.walk / 2 : step;
4      return { group: 'crouch8', index: cycle(gait, 8) };
5  }
6      // Frame 13 is a crawl, not a recoil cel. Do not swap the entire body.
7      if (u.fire > 0 || u.secondaryFire > 0) return action(1);
8      if ((u.aimUntil ?? 0) > time) return action(1);
9      return action(1);
10 }
11 if (u.pose === 'hunker') {
12     if (u.moving) return { group: 'crouch8', index: cycle(step, 8) };
13     // Hunker is the same committed knee-height class, not periodic crawling.
14     return action(1);
15 }
16 if (u.moving)
17     return u.pose === 'run' || u.tactic === 'retreat'
18         ? action(16 + cycle(u.walk / 2, 4))
19         : { group: 'walk8', index: cycle(step, 8) };
20 // v105: hit flinches — a tall stagger, a knee-buck and a deep cower-flinch
21 // — so a burst walking across a squad doesn't pop the identical frame on
22 // every man. v116 widened the spread from two to three variants, v118 to
23 // four — the deep curl joins the rotation. v120: the flinch now cycles
24 // across the full 0.16s hit window instead of flashing for 0.03s, so a
25 // burst reads as a live reaction chain — stagger, knee-soft, curl, squat —
26 // rather than a single popped frame.
27 if (u.flash > 0) {
28     return reaction(4);
29 }
30 // A stationary rifleman keeps the aimed stance (action 0) while firing —
31 // the renderer's patrol layer overlays the dedicated aimed-rifle pose
32 // (raise3[2]) for the whole burst, so the weapon reads as shouldered and
33 // on target between shots instead of rocking through gait frames.
34 // v117: underslung / personal secondary discharge — the arm-forward frame
35 // for the 0.09s window so a GL or pistol shot reads as its own beat
36 // instead of vanishing under the patrol idle. Non-plain on purpose: it
37 // makes the patrol layer yield so the frame actually shows.
38 if (
39     u.secondaryFire > 0 &&
40     !u.moving &&
41     (u.pose === 'idle' || u.pose === 'walk')
42 )
43     // v128: point-forward signal beat instead of the climb pose.
44     return action(0);
45 return action(0);
46 }
47 export function adultWreckChoice(
48     age: number,
49     pose?: Unit['pose'],
50     seed = 0,

```

game/adult-animation.ts:715 至 game/adult-animation.ts:764

```

1 ): AdultFrameChoice {
2   if (pose === 'prone') return action(15);
3   // v105: visibly different death throes so a field of casualties doesn't
4   // play the same stagger in unison. v116 widened the spread from three to
5   // six variants — classic stagger, knee crumple, clean drop, forward pitch,
6   // slow sink and a spin — so a platoon's worth of wrecks rarely repeats.
7   // v120 widened to eight — a double-take stagger and a slow fold join the
8   // rotation.
9   // The seed is the casualty's uid, stable for the wreck's whole lifetime.
10  const variant = seed % 8;
11  if (variant === 2) return action(15); // clean drop: killed mid-stride
12  if (pose === 'crouch' || pose === 'hunker' || pose === 'land')
13    return age < 0.45
14      ? reaction(5 + Math.min(2, Math.floor(age * 6)))
15      : action(15);
16  if (variant === 1)
17    // Crumple: buckle at the knees first, then go down.
18    return age < 0.5
19      ? reaction(5 + Math.min(1, Math.floor(age * 4)))
20      : action(15);
21  if (variant === 3)
22    // Forward pitch: flinch, then pitch onto the face.
23    return age < 0.55
24      ? age < 0.25
25        ? reaction(4)
26        : reaction(7)
27      : action(15);
28  if (variant === 4)
29    // Slow sink: knees give out slowly, then the body folds.
30    return age < 0.7 ? (age < 0.4 ? reaction(5) : reaction(7)) : action(15);
31  if (variant === 5)
32    // Spin: twist under the impact, stagger, then drop.
33    return age < 0.6
34      ? [reaction(6), reaction(4), reaction(7)][
35        Math.min(2, Math.floor(age * 5))
36      ]
37      : action(15);
38  if (variant === 6)
39    // v120: double-take stagger — flinch, curl, deep squat before dropping.
40    return age < 0.55
41      ? [reaction(4), reaction(6), reaction(7)][
42        Math.min(2, Math.floor(age * 5))
43      ]
44      : action(15);
45  if (variant === 7)
46    // v120: slow fold — curl, knee-soft, then down.
47    return age < 0.7 ? (age < 0.35 ? reaction(6) : reaction(5)) : action(15);
48  // Classic: hit stagger, stumble, drop.
49  return age < 0.6
50    ? reaction(4 + Math.min(3, Math.floor(age * 6)))

```

game/adult-animation.ts:765 至 game/adult-atlas.ts:39

```

1      : action(15);
2  }
3  /**
4   * v106: a soldier thrown by a blast. While airborne the body cycles the
5   * stagger/flinch frames fast so the tumble reads as a man flung through the
6   * air; the renderer spins the sprite on the wreck's spin axis. On landing the
7   * wreck switches to adultWreckChoice's final prone frame.
8   */
9  export function ragdollChoice(age: number, seed = 0): AdultFrameChoice {
10    return reaction(4 + ((Math.floor(age * 12) + (seed % 2)) % 4));
11  }
12  import { transparentSheet } from './sprite-atlas';
13  import type { AdultSprites } from './adult-animation';
14  /** The packed reference already uses a 384px cell, 252px adult and a shared foot anchor. */
15  export function adultAtlas(image: HTMLImageElement): Omit<AdultSprites, 'signals4' | 'reload8' | 'grenade8' | 'stance16' | 'lowReload16' | 'medical24' | 'lowGrenade32' | 'repair34' | 'proneIdle8'> {
16    const source = transparentSheet(image),
17      context = source.getContext('2d')!;
18    const pixels = context.getImageData(0, 0, source.width, source.height);
19    for (let at = 3; at < pixels.data.length; at += 4)
20      pixels.data[at] = pixels.data[at] > 120 ? 255 : 0;
21    context.putImageData(pixels, 0, 0);
22    const cellW = source.width / 8,
23      cellH = source.height / 6;
24    const frames = Array.from({ length: 44 }, (_, index) => {
25      const out = document.createElement('canvas');
26      out.width = out.height = 96;
27      const ctx = out.getContext('2d')!;
28      ctx.imageSmoothingEnabled = false;
29      // Do not fit individual bounding boxes: crouching and prone bodies keep their scale.
30      ctx.drawImage(
31        source,
32        (index % 8) * cellW,
33        Math.floor(index / 8) * cellH,
34        cellW,
35        cellH,
36        0,
37        0,
38        96,
39        96,
40      );
41      return out;
42    });
43    return {
44      walk8: frames.slice(0, 8),
45      crouch8: frames.slice(8, 16),
46      actions20: frames.slice(16, 36),
47      // Save the authored pair BEFORE loadArt replaces actions20[2] with the
48      // new, lower settled stance. Mixing that 17px body into this 23–25px
49      // crawl made every step visibly flatten and spring up again.
50      crawl2: [frames[18], frames[28]],

```

game/adult-atlas.ts:40 至 game/adult-atlas.ts:92

```

1   reactions8: frames.slice(36, 44),
2   };
3 }
4 /** Generated eight-cel standing reload, measured rather than square-grid fitted.
5  * Common scale retains the 63px adult and a fixed 95px boot anchor. */
6 export function standingReloadFrames(image: HTMLImageElement): HTMLCanvasElement[] {
7   const source = transparentSheet(image);
8   const cellW = source.width / 4, cellH = source.height / 2;
9   const scale = 63 / 454;
10  return Array.from({ length: 8 }, (_, i) => {
11    const row = Math.floor(i / 4), out = document.createElement('canvas');
12    out.width = out.height = 96;
13    const ctx = out.getContext('2d')!;
14    ctx.imageSmoothingEnabled = true;
15    const baseline = row === 0 ? 578 : 565;
16    ctx.drawImage(source, (i % 4) * cellW, row * cellH, cellW, cellH,
17      48 - 130 * scale, 95 - baseline * scale, cellW * scale, cellH * scale);
18    return out;
19  });
20 }
21 /** Measured gutters, NOT equal cells: the release hand extends past x=314.
22  * Registration still uses the original common pelvis/boot anchors, not each
23  * silhouette's bounding box, so an outstretched arm cannot shrink the body. */
24 export function standingGrenadeFrames(image: HTMLImageElement): HTMLCanvasElement[] {
25   const source = transparentSheet(image), scale = 63 / 504;
26   const cuts = [[40, 315, 640, 970, 1240], [40, 355, 640, 970, 1240]];
27   return Array.from({ length: 8 }, (_, i) => {
28     const row = Math.floor(i / 4), col = i % 4;
29     const left = cuts[row][col], right = cuts[row][col + 1];
30     const out = document.createElement('canvas');
31     out.width = out.height = 96;
32     const ctx = out.getContext('2d')!;
33     ctx.imageSmoothingEnabled = true;
34     ctx.drawImage(source, left, row * 627 + 70, right - left, 528,
35       48 + (left - col * 313.5 - 171) * scale,
36       95 + (70 - 587) * scale, (right - left) * scale, 528 * scale);
37     return out;
38   });
39 }
40 /**
41  * v121: dedicated hand-signal sheets (2x2 grid, point-forward / wave-overhead /
42  * point-back / fist). The old signal branch reused climbing frames, which read
43  * as the leader hauling himself up a rope instead of directing the squad.
44  * These sheets are painted at high resolution with real anti-aliased alpha, so
45  * unlike adultAtlas they keep smoothing on and skip the alpha binarisation.
46  */
47 export function signalFrames(image: HTMLImageElement): HTMLCanvasElement[] {
48   const source = transparentSheet(image);
49   const cellW = source.width / 2,
50   cellH = source.height / 2;

```

game/adult-atlas.ts:93 至 game/adult-specialists.ts:7

```

1   return Array.from({ length: 4 }, (_, index) => {
2     const out = document.createElement('canvas');
3     out.width = out.height = 96;
4     const ctx = out.getContext('2d')!;
5     ctx.imageSmoothingEnabled = true;
6     ctx.drawImage(
7       source,
8       (index % 2) * cellW,
9       Math.floor(index / 2) * cellH,
10      cellW,
11      cellH,
12      0,
13      0,
14      96,
15      96,
16    );
17    return out;
18  });
19 }
20 /**
21  * v174: per-identity pose chain built from the soldier's OWN pixel atlas, so
22  * a yellow marine or militia fighter never snaps to the shared green realistic
23  * stance sheet when aiming or dropping to a knee. Index semantics match the
24  * legacy painted stance16: 0 standing aim, 7 kneeling aim, 15 prone aim,
25  * 8 = 7 handoff. Intermediate indices reuse the closest authored height from
26  * the same atlas (crouch-walk for the stand->knee descent, prone-rise and
27  * crawl for the knee->prone descent), and the travel endpoints hand off
28  * seamlessly to the crouch8 / crawl2 gait cycles.
29  */
30 export function ownStance16(
31   adult: ReturnType<typeof adultAtlas>,
32 ): HTMLCanvasElement[] {
33   const stand = adult.actions20[0];
34   const knee = adult.actions20[1];
35   const prone = adult.actions20[2];
36   const crouchWalk = adult.crouch8[0];
37   const crawlAlt = adult.crawl2[1];
38   const proneRise = adult.actions20[13];
39   return [
40     stand, stand, crouchWalk, crouchWalk, knee, knee, knee, knee,
41     knee, proneRise, proneRise, crawlAlt, prone, prone, prone, prone,
42   ];
43 }
44 import { weaponModel } from './cards';
45 import type { Unit } from './engine';
46 import type { AdultFrameChoice } from './adult-animation';
47 import { SPECIALIST_LANDMARKS } from './specialist-landmarks';
48 import { isHeavyGunner } from './machinegun-team';
49 import { isPrecisionObserver } from './precision-team';
50 import { specialistLegWaist, specialistBodyOffset } from './weapon-pose-data';

```

game/adult-specialists.ts:8 至 game/adult-specialists.ts:57

```
1  export type SpecialistRole =
2    | 'machinegun'
3    | 'rocket'
4    | 'grenade'
5    | 'sniper'
6    | 'medic'
7    | 'mortar';
8  export interface SpecialistFrame {
9    image: HTMLCanvasElement;
10   waist: readonly number[];
11   muzzle: readonly number[] | null;
12 }
13 export interface AdultSpecialistArt {
14   standing: SpecialistFrame;
15   crouch: SpecialistFrame;
16   prone: SpecialistFrame;
17   /** A whole-body painted drill, including its exact settled endpoints. */
18   stance16?: SpecialistFrame[];
19 }
20 export type AdultSpecialists = Partial<
21   Record<SpecialistRole, AdultSpecialistArt>
22 >;
23 export interface SpecialistSprite {
24   image: HTMLCanvasElement;
25   muzzle: { x: number; height: number } | null;
26 }
27 const roles: SpecialistRole[] = [
28   'machinegun',
29   'rocket',
30   'sniper',
31   'medic',
32   'mortar',
33 ];
34 function canvas() {
35   const out = document.createElement('canvas');
36   out.width = 128;
37   out.height = 96;
38   return out;
39 }
40 /** The prepared sheet preserves adult anatomy; no per-pose fitting or weapon drawing. */
41 export function specialistAtlas(source: HTMLImageElement): AdultSpecialists {
42   const sets: AdultSpecialists = {};
43   for (let col = 0; col < 5; col++) {
44     const rows = [0, 1, 2].map((row) => {
45       const image = canvas(),
46         ctx = image.getContext('2d')!;
47       ctx.imageSmoothingEnabled = false;
48       ctx.drawImage(
49         source,
50         (col * source.width) / 5,
```

game/adult-specialists.ts:58 至 game/adult-specialists.ts:107

```

1      (row * source.height) / 3,
2      source.width / 5,
3      source.height / 3,
4      0,
5      0,
6      128,
7      96,
8      );
9      return { image, ...SPECIALIST_LANDMARKS[row * 5 + col] };
10     });
11     sets[roles[col]] = { standing: rows[0], crouch: rows[1], prone: rows[2] };
12   }
13   return sets;
14 }
15 const cache = new WeakMap<
16   HTMLCanvasElement,
17   Map<SpecialistFrame, SpecialistSprite>
18 >();
19 /** Replace the baked rifle torso, retaining only the generated moving legs. */
20 export function specialistSprite(
21   base: HTMLCanvasElement,
22   choice: AdultFrameChoice,
23   u: Unit,
24   sets?: AdultSpecialists,
25   weaponStances?: AdultSpecialists,
26 ): SpecialistSprite | null {
27   const model = (u.id === 'grenadiers' ? 'grenade' : weaponModel(u)) as SpecialistRole,
28     // A portable LMG is not the heavy tripod gun. The precision observer
29     // carries ordinary kit/radio, not a second sniper rifle.
30     set =
31       (isHeavyGunner(u) && !isPrecisionObserver(u)
32         ? weaponStances?.[model]
33         : undefined) ?? sets?.[model];
34   if (
35     !set ||
36     isPrecisionObserver(u) ||
37     u.hp <= 0 ||
38     u.wounded ||
39     u.surrendered ||
40     choice.group === 'reactions8'
41   )
42     return null;
43   if (choice.group === 'stance16') {
44     // Stance transitions use the identity's own per-identity stance16 frames
45     // (built by ownStance16), which carry the correct uniform colour. The
46     // generic green weapon-stance sheet would repaint the whole body and
47     // shrink it (128×96 vs 96×96), so yield to the identity art here.
48     return null;
49   }
50   const prone = choice.group === 'crawl2' ||

```

game/adult-specialists.ts:108 至 game/adult-specialists.ts:157

```
1    choice.group === 'actions20' && [2, 12].includes(choice.index);
2    const low =
3    choice.group === 'crouch8' ||
4    (choice.group === 'actions20' && choice.index === 1);
5    const running =
6    choice.group === 'actions20' && choice.index >= 16 && choice.index <= 19;
7    if (
8    !prone &&
9    !low &&
10   !running &&
11   choice.group !== 'walk8' &&
12   !(choice.group === 'actions20' && choice.index === 0)
13  )
14    return null;
15    const part = prone
16    ? set.prone
17    : low && !(model === 'mortar' && choice.group === 'crouch8')
18    ? set.crouch
19    : set.standing;
20    // Static poses (standing aim / crouch / prone) used to return the full
21    // generic green 128×96 body, which repainted the soldier in the wrong
22    // uniform and changed the apparent size. Always composite the specialist
23    // weapon torso onto the identity's own legs so the uniform colour and
24    // silhouette stay correct.
25    let variants = cache.get(base);
26    if (!variants) {
27      variants = new Map();
28      cache.set(base, variants);
29    }
30    const found = variants.get(part);
31    if (found) return found;
32    const waist = specialistLegWaist(choice.group, choice.index);
33    const image = canvas(),
34      ctx = image.getContext('2d')!,
35      ctx.imageSmoothingEnabled = false;
36    const {x:dx,y:dy}=specialistBodyOffset(choice.group, choice.index, part);
37    const targetX = 16 + waist[0],
38      targetY = waist[1];
39    ctx.save();
40    ctx.beginPath();
41    if (prone) ctx.rect(0, 0, targetX + 1, 96);
42    else ctx.rect(0, targetY, 128, 96 - targetY);
43    ctx.clip();
44    ctx.drawImage(base, 16, 0);
45    ctx.restore();
46    ctx.save();
47    ctx.beginPath();
48    if (prone) {
49      // Retain the complete generic legs and the specialist's rearward launcher/pack.
50      // Only the truncated source pants stub is excluded.
```

game/adult-specialists.ts:158 至 game/ambience.ts:34

```

1   ctx.rect(Math.round(part.waist[0]) - 1 + dx, 0, 128, 96);
2   ctx.rect(
3     0,
4     0,
5     Math.round(part.waist[0]) + dx,
6     Math.round(part.waist[1]) - 3 + dy,
7   );
8 } else ctx.rect(0, 0, 128, Math.round(part.waist[1]) + 1 + dy);
9 ctx.clip();
10 ctx.drawImage(part.image, dx, dy);
11 ctx.restore();
12 const result = {
13   image,
14   muzzle: part.muzzle
15   ? { x: part.muzzle[0] + dx - 64, height: 96 - part.muzzle[1] - dy }
16   : null,
17 };
18 variants.set(part, result);
19 return result;
20 }
21 import { CARDS, W, ground, pointVisible, veteranTier, type DragMark, type GameState, type Scorch, type TreadMark, type
  Unit } from './engine';
22 import { drawSmokePuff } from './effect-atlas';
23 /** Deterministic hash  $\rightarrow [0,1)$  */
24 function hash(n: number): number {
25   const x = Math.sin(n * 127.1 + 311.7) * 43758.5453;
26   return x - Math.floor(x);
27 }
28 // — Birds —————
29 const FLOCK_PERIOD = 27; // seconds between flocks
30 const FLOCK_CROSS = 11; // seconds for a flock to cross the map
31 /**
32  * Small flocks cross the sky, suggesting a living world beyond the trenches.
33  * Purely decorative — drawn in world space so the camera transform applies.
34  */
35 export function drawBirds(
36   ctx: CanvasRenderingContext2D,
37   s: GameState,
38   camera: number,
39   viewportWidth: number,
40 ) {
41   const t = s.time;
42   const cycle = t % FLOCK_PERIOD;
43   if (cycle >= FLOCK_CROSS) return;
44   const flockIdx = Math.floor(t / FLOCK_PERIOD);
45   const seed = flockIdx * 91.7;
46   const dir = hash(seed) > 0.5 ? 1 : -1;
47   const progress = cycle / FLOCK_CROSS;
48   const startX = dir > 0 ? -60 : W + 60;
49   const endX = dir > 0 ? W + 60 : -60;
50   const cx = startX + (endX - startX) * progress;

```

game/ambience.ts:35 至 game/ambience.ts:88

```

1  const baseY = 28 + hash(seed + 1) * 52;
2  const count = 4 + Math.floor(hash(seed + 2) * 5);
3  ctx.strokeStyle = 'rgba(35,38,33,0.45)';
4  ctx.lineWidth = 1;
5  ctx.beginPath();
6  for (let i = 0; i < count; i++) {
7      const ox = (i - count / 2) * 9;
8      const bx = cx + ox * dir;
9      if (bx < camera - 20 || bx > camera + viewportWidth + 20) continue;
10     const by =
11         baseY + Math.abs(ox) * 0.32 + Math.sin(t * 2.5 + i * 1.3) * 1.5;
12     const flap = Math.sin(t * 9 + i * 2.1) * 1.5;
13     ctx.moveTo(bx - 2.5, by + flap);
14     ctx.lineTo(bx, by - flap * 0.6);
15     ctx.lineTo(bx + 2.5, by + flap);
16 }
17 ctx.stroke();
18 }
19 // — Distant horizon flashes —————
20 const FLASH_PERIOD = 18;
21 const FLASH_DURATION = 0.55;
22 /**
23  * State of the distant-flash cycle at time t. Shared by the renderer and the
24  * audio layer so the delayed thunder stays in sync with the horizon glow.
25  */
26 export function distantFlashState(t: number): {
27     active: boolean;
28     x: number;
29     index: number;
30 } {
31     const index = Math.floor(t / FLASH_PERIOD);
32     const cycle = t % FLASH_PERIOD;
33     if (cycle >= FLASH_DURATION) return { active: false, x: 0, index };
34     const fseed = index * 57.3;
35     const side = hash(fseed) > 0.5 ? 0 : 1;
36     const x =
37         side === 0
38             ? 24 + hash(fseed + 1) * 90
39             : W - 24 - hash(fseed + 1) * 90;
40     return { active: true, x, index };
41 }
42 /**
43  * Brief flashes at the far left/right edges suggest a larger battle
44  * happening just beyond the playable area.
45  */
46 export function drawDistantFlashes(
47     ctx: CanvasRenderingContext2D,
48     s: GameState,
49     camera: number,
50     viewportWidth: number,

```

game/ambience.ts:89 至 game/ambience.ts:145

```

1  ){
2      const t = s.time;
3      const flash = distantFlashState(t);
4      if (!flash.active) return;
5      const fx = flash.x;
6      if (fx < camera - 60 || fx > camera + viewportWidth + 60) return;
7      const cycle = t % FLASH_PERIOD;
8      const fseed = flash.index * 57.3;
9      const fy = ground(s, fx) - 55 - hash(fseed + 2) * 45;
10     const intensity = 1 - cycle / FLASH_DURATION;
11     const r = 14 + hash(fseed + 3) * 20;
12     const grad = ctx.createRadialGradient(fx, fy, 0, fx, fy, r);
13     grad.addColorStop(0, `rgba(255,210,130,${0.45 * intensity})`);
14     grad.addColorStop(0.4, `rgba(255,160,70,${0.18 * intensity})`);
15     grad.addColorStop(1, 'rgba(255,100,30,0)');
16     ctx.fillStyle = grad;
17     ctx.fillRect(fx - r, fy - r, r * 2, r * 2);
18 }
19 // — Wreck smoke columns —————
20 const SMOKE_PUFF_INTERVAL = 0.38;
21 const SMOKE_PUFF_LIFE = 4.0;
22 const SMOKE_COLUMN_HEIGHT = 62;
23 /**
24  * Destroyed vehicles bleed slow smoke columns that drift with the wind.
25  * Fresh wrecks smoke heavily; the column thins over two minutes.
26  */
27 export function drawWreckSmoke(
28     ctx: CanvasRenderingContext2D,
29     s: GameState,
30     camera: number,
31     viewportWidth: number,
32     smokeFrames?: HTMLCanvasElement[],
33 ){
34     const now = s.time;
35     for (const w of s.wrecks) {
36         const c = CARDS[w.cardId];
37         if (w.abandoned || w.cardId==='glider_transport' || (!c.armored && !c.vehicle)) continue;
38         if (w.x < camera - 80 || w.x > camera + viewportWidth + 80) continue;
39         const strength = Math.max(0, 1 - w.age / 120);
40         if (strength <= 0.02) continue;
41         const baseY = w.y - 10;
42         const seedBase = w.id * 13.7;
43         // A newly destroyed vehicle has no smoke history. Using global match time
44         // spawned an entire four-second column high in the sky on its first frame.
45         const latestPuff = Math.floor(w.age / SMOKE_PUFF_INTERVAL);
46         const maxPuffs = Math.ceil(SMOKE_PUFF_LIFE / SMOKE_PUFF_INTERVAL);
47         for (let p = 0; p < maxPuffs; p++) {
48             const puffIdx = latestPuff - p;
49             if (puffIdx < 0) continue;
50             const puffStart = puffIdx * SMOKE_PUFF_INTERVAL;

```

game/ambience.ts:146 至 game/ambience.ts:200

```
1    const age = w.age - puffStart;
2    if (age > SMOKE_PUFF_LIFE) continue;
3    const lifeT = age / SMOKE_PUFF_LIFE;
4    const rise = lifeT * SMOKE_COLUMN_HEIGHT;
5    const sway =
6      Math.sin(now * 0.7 + puffIdx * 1.3) * 3 * lifeT +
7      s.wind * age * 0.6;
8    const jitter = (hash(seedBase + puffIdx * 7.1) - 0.5) * 6;
9    const px = w.x + sway + jitter;
10   const py = baseY - rise;
11   const size = Math.round(
12     (3 + lifeT * 12) * (0.65 + strength * 0.35),
13   );
14   const alpha = (1 - lifeT) * 0.3 * strength;
15   if (alpha < 0.015) continue;
16   if (smokeFrames) {
17     drawSmokePuff(ctx, smokeFrames, lifeT, px, py, size * 2.2, '#44423c', alpha);
18     continue;
19   }
20   ctx.fillStyle = `rgba(68,66,60,${alpha.toFixed(3)})`;
21   // Two overlapping rects → soft blob, matching the particle style.
22   ctx.fillRect(
23     Math.round(px - size / 2),
24     Math.round(py - size / 3),
25     size,
26     Math.max(1, Math.round(size * 0.65)),
27   );
28   ctx.fillRect(
29     Math.round(px - size / 3),
30     Math.round(py - size / 2),
31     Math.max(1, Math.round(size * 0.65)),
32     size,
33   );
34 }
35 }
36 }
37 // — Scorch marks —————
38 function drawScorch(ctx: CanvasRenderingContext2D, sc: Scorch, gy: number) {
39   let seed = sc.seed;
40   const rnd = () => {
41     seed = (Math.imul(1664525, seed) + 1013904223) >>> 0;
42     return seed / 4294967296;
43   };
44   const r = sc.radius;
45   ctx.save();
46   // Core dark patch
47   ctx.fillStyle = '#1c1712';
48   ctx.globalAlpha = 0.42;
49   const core = 6 + Math.floor(rnd() * 5);
50   for (let i = 0; i < core; i++) {
```

game/ambience.ts:201 至 game/ambience.ts:251

```

1    const a = rnd() * Math.PI * 2;
2    const d = rnd() * r * 0.45;
3    const w = r * (0.3 + rnd() * 0.4);
4    const h = Math.max(1, Math.round(w * 0.4));
5    ctx.fillRect(
6      Math.round(sc.x + Math.cos(a) * d - w / 2),
7      Math.round(gy - h / 2 - 1),
8      Math.round(w),
9      h,
10   );
11 }
12 // Burnt rim
13 ctx.fillStyle = '#33281e';
14 ctx.globalAlpha = 0.3;
15 const rim = 5 + Math.floor(rnd() * 4);
16 for (let i = 0; i < rim; i++) {
17   const a = rnd() * Math.PI * 2;
18   const d = r * (0.35 + rnd() * 0.4);
19   const w = r * (0.15 + rnd() * 0.2);
20   ctx.fillRect(
21     Math.round(sc.x + Math.cos(a) * d - w / 2),
22     Math.round(gy - 2),
23     Math.round(w),
24     2,
25   );
26 }
27 // Scattered char flecks
28 ctx.fillStyle = '#241d18';
29 ctx.globalAlpha = 0.35;
30 const flecks = 8 + Math.floor(rnd() * 8);
31 for (let i = 0; i < flecks; i++) {
32   const a = rnd() * Math.PI * 2;
33   const d = r * (0.5 + rnd() * 0.55);
34   ctx.fillRect(
35     Math.round(sc.x + Math.cos(a) * d),
36     Math.round(gy - 1 - rnd() * 2),
37     2,
38     1,
39   );
40 }
41 ctx.restore();
42 }
43 /** Persistent blast scorches drawn on the ground, under units and props. */
44 export function drawScorches(
45   ctx: CanvasRenderingContext2D,
46   s: GameState,
47   camera: number,
48   viewportWidth: number,
49 ) {
50   for (const sc of s.scorches) {

```

game/ambience.ts:252 至 game/ambience.ts:304

```
1     if (sc.x < camera - 80 || sc.x > camera + viewportWidth + 80) continue;
2     drawScorch(ctx, sc, ground(s, sc.x));
3 }
4 }
5 // — Vehicle tread marks —————
6 const TREAD_LIFETIME = 40; // seconds before a mark fully fades
7 function drawTread(
8   ctx: CanvasRenderingContext2D,
9   t: TreadMark,
10  gy: number,
11  age: number,
12 ){
13   const fade = Math.max(0, 1 - age / TREAD_LIFETIME);
14   if (fade <= 0) return;
15   let seed = t.seed;
16   const rnd = () => {
17     seed = (Math.imul(1664525, seed) + 1013904223) >>> 0;
18     return seed / 4294967296;
19   };
20   const spacing = t.half * 0.88; // track centres sit just inside the hull edge
21   const bandW = 7; // track width, px
22   const bandL = 15; // mark length along the direction of travel
23   ctx.save();
24   for (const side of [-1, 1]) {
25     const bx = t.x + side * spacing;
26     // Compressed earth band
27     ctx.fillStyle = '#241f19';
28     ctx.globalAlpha = 0.3 * fade;
29     ctx.fillRect(
30       Math.round(bx - bandL / 2),
31       Math.round(gy - bandW / 2),
32       bandL,
33       bandW,
34     );
35     // Slight ridge at the leading edge
36     ctx.fillStyle = '#3a322a';
37     ctx.globalAlpha = 0.18 * fade;
38     ctx.fillRect(
39       Math.round(bx - bandL / 2),
40       Math.round(gy - bandW / 2 - 1),
41       bandL,
42       1,
43     );
44     // Texture flecks
45     ctx.fillStyle = '#1a1612';
46     ctx.globalAlpha = 0.22 * fade;
47     const flecks = 3 + Math.floor(rnd() * 3);
48     for (let i = 0; i < flecks; i++) {
49       ctx.fillRect(
50         Math.round(bx - bandL / 2 + rnd() * bandL),
```

minigame/src/ui/lobby.ts:101 至 minigame/src/ui/lobby.ts:155

```
1      { page: 'settings', label: '设置' },
2      { page: 'guide', label: '作战手册' },
3    ];
4    let y = 64;
5    for (const item of nav) {
6      const btn = new Button(item.label, SIDEBAR_W - 16, 32, {
7        fontSize: 13,
8        bg: COLORS.bgPanel,
9      });
10     btn.x = 8;
11     btn.y = y;
12     btn.onTap = () => {
13       if (item.page === 'builder') this.router.navigate('deck-builder');
14       else if (item.page === 'shop') this.router.navigate('shop');
15       else this.setSubpage(item.page as LobbyPage);
16     };
17     this.addChild(btn);
18     this.navButtons.push(btn);
19     y += 36;
20   }
21   y += 4;
22   this.mapBtn = new Button("", SIDEBAR_W - 16, 44, { fontSize: 11 });
23   this.mapBtn.x = 8;
24   this.mapBtn.y = y;
25   this.mapBtn.onTap = () => {
26     const idx = MAP_IDS.indexOf(lobbyState.mapId);
27     const next = MAP_IDS[(idx + 1) % MAP_IDS.length];
28     lobbyState.setMap(next);
29     this.refreshSidebarLabels();
30   };
31   this.addChild(this.mapBtn);
32   y += 50;
33   this.nightBtn = new Button("", SIDEBAR_W - 16, 40, { fontSize: 11 });
34   this.nightBtn.x = 8;
35   this.nightBtn.y = y;
36   this.nightBtn.onTap = () => {
37     lobbyState.setNight(!lobbyState.night);
38     this.refreshSidebarLabels();
39   };
40   this.addChild(this.nightBtn);
41   y += 46;
42   this.diffBtn = new Button("", SIDEBAR_W - 16, 40, { fontSize: 11 });
43   this.diffBtn.x = 8;
44   this.diffBtn.y = y;
45   this.diffBtn.onTap = () => this.setSubpage('settings');
46   this.addChild(this.diffBtn);
47   this.refreshSidebarLabels();
48 }
49 private refreshSidebarLabels(): void {
50   const map = MAPS[lobbyState.mapId];
```

minigame/src/ui/lobby.ts:156 至 minigame/src/ui/lobby.ts:211

```
1   this.mapBtn.label = `地图: ${map.name}`;
2   this.nightBtn.label = lobbyState.night ? '时段: 夜间' : '时段: 昼间';
3   this.diffBtn.label = `对手: ${DIFFICULTY_LABEL[lobbyState.difficulty]}`;
4 }
5 // — Content pages —————
6 private setSubpage(page: LobbyPage): void {
7   this.subpage = page;
8   this.scrollY = 0;
9   this.buildContent();
10  // Map/night selectors hidden on campaign (mission defines its own map)
11  this.mapBtn.visible = page !== 'campaign';
12  this.nightBtn.visible = page !== 'campaign';
13 }
14 private clearContent(): void {
15   const keep = new Set<Widget>([
16     ...this.navButtons,
17     this.mapBtn,
18     this.nightBtn,
19     this.diffBtn,
20   ]);
21   for (const c of [...this.children]) {
22     if (!keep.has(c)) this.removeChild(c);
23   }
24 }
25 private buildContent(): void {
26   this.clearContent();
27   if (this.subpage === 'home') this.buildHome();
28   else if (this.subpage === 'settings') this.buildSettings();
29   else if (this.subpage === 'campaign') this.buildCampaign();
30   // guide is pure text, drawn in drawSelf
31 }
32 private buildHome(): void {
33   this.startBtn = new Button('开始游戏', 220, 56, {
34     fontSize: 20,
35     bold: true,
36     bg: COLORS.accent,
37     bgPressed: COLORS.accentDark,
38     textColor: '#1a1f1a',
39     border: COLORS.accentDark,
40     radius: 6,
41   });
42   this.startBtn.x = this.contentX() + (this.contentW() - 220) / 2;
43   this.startBtn.y = this.screenH - 96;
44   this.startBtn.onTap = () => {
45     const err = lobbyState.begin();
46     if (err) this.toast(err);
47   };
48   this.addChild(this.startBtn);
49 }
50 private buildSettings(): void {
```

minigame/src/ui/lobby.ts:212 至 minigame/src/ui/lobby.ts:265

```
1    const x = this.contentX() + 24;
2    let y = 24;
3    // Difficulty buttons
4    const diffs: Difficulty[] = ['standard', 'veteran', 'elite'];
5    const bw = 150;
6    for (const d of diffs) {
7        const btn = new Button(
8            `${DIFFICULTY_LABEL[d]}\n${DIFFICULTY_BONUS[d]}`,
9            bw,
10           52,
11           {
12               fontSize: 12,
13               bg:
14                   lobbyState.difficulty === d ? COLORS.green : COLORS.bgPanelLight,
15               border:
16                   lobbyState.difficulty === d ? COLORS.greenLight : COLORS.border,
17           },
18        );
19        btn.x = x + diffs.indexOf(d) * (bw + 10);
20        btn.y = y;
21        const dd = d;
22        btn.onTap = () => {
23            lobbyState.setDifficulty(dd);
24            this.buildContent();
25            this.refreshSidebarLabels();
26        };
27        this.addChild(btn);
28    }
29    y += 66;
30    // Sound toggle
31    const audio = getBattleAudio().settings;
32    const toggle = new Toggle(44, 22, audio.enabled);
33    toggle.x = x;
34    toggle.y = y;
35    toggle.onChange = (v) => {
36        getBattleAudio().configure({ enabled: v });
37    };
38    this.addChild(toggle);
39    y += 36;
40    // Music volume
41    const musicSlider = new Slider(260, 24, audio.music);
42    musicSlider.x = x;
43    musicSlider.y = y;
44    musicSlider.onChange = (v) => {
45        getBattleAudio().configure({ music: v });
46    };
47    this.addChild(musicSlider);
48    y += 36;
49    // SFX volume
50    const sfxSlider = new Slider(260, 24, audio.effects);
```

minigame/src/ui/lobby.ts:266 至 minigame/src/ui/lobby.ts:317

```
1   sfxSlider.x = x;
2   sfxSlider.y = y;
3   sfxSlider.onChange = (v) => {
4     getBattleAudio().configure({ effects: v });
5   };
6   this.addChild(sfxSlider);
7 }
8 private buildCampaign(): void {
9   const x = this.contentX() + 16;
10  let y = 16;
11  for (let i = 0; i < MISSIONS.length; i++) {
12    const m = MISSIONS[i];
13    const done = lobbyState.completed.includes(m.id);
14    const unlocked = lobbyState.isMissionUnlocked(i);
15    const btn = new Button(
16      `${m.chapter} ${m.title} ${done ? '✓' : unlocked ? " : '□'}`,
17      150,
18      40,
19      {
20        fontSize: 11,
21        bg:
22          this.selectedMission === m.id
23            ? COLORS.green
24            : COLORS.bgPanel,
25      },
26    );
27    btn.x = x;
28    btn.y = y;
29    btn.enabled = unlocked;
30    btn.onTap = () => {
31      this.selectedMission = m.id;
32      this.buildContent();
33    };
34    this.addChild(btn);
35    y += 46;
36  }
37  // Start button
38  const idx = MISSIONS.findIndex((m) => m.id === this.selectedMission);
39  const unlocked = lobbyState.isMissionUnlocked(idx);
40  const startBtn = new Button(
41    unlocked ? '进入战役 →' : '完成上一章后解锁',
42    170,
43    40,
44    {
45      fontSize: 13,
46      bold: true,
47      bg: unlocked ? COLORS.accent : COLORS.bgPanelLight,
48      textColor: unlocked ? '#1a1f1a' : COLORS.textMuted,
49    },
50  );
```

minigame/src/ui/lobby.ts:318 至 minigame/src/ui/lobby.ts:375

```
1     startBtn.x = x;
2     startBtn.y = y + 8;
3     startBtn.enabled = unlocked;
4     startBtn.onTap = () => {
5         const err = lobbyState.begin(undefined, this.selectedMission);
6         if (err) this.toast(err);
7     };
8     this.addChild(startBtn);
9 }
10 // — Images —————
11 private loadHomeBg(): void {
12     const img = new Image();
13     img.onload = () => {
14         this.homeBg = img;
15     };
16     img.src = assetUrl('/art/home-camp-v14.png');
17 }
18 private loadMissionBg(url: string): void {
19     if (this.missionBgUrl === url) return;
20     this.missionBgUrl = url;
21     this.missionBg = null;
22     const img = new Image();
23     img.onload = () => {
24         if (this.missionBgUrl === url) this.missionBg = img;
25     };
26     img.src = assetUrl(url);
27 }
28 // — Toast —————
29 private toast(msg: string): void {
30     this.toastMsg = msg;
31     this.toastTime = Date.now();
32 }
33 // — Lifecycle —————
34 onEnter(): void {
35     this.refreshSidebarLabels();
36     this.buildContent();
37     const mixer = getBattleAudio();
38     mixer.prefetch();
39     mixer.setActive(true);
40 }
41 handleTouch(ev: TouchEvent): boolean {
42     if (!this.audioUnlocked && ev.type === 'down') {
43         this.audioUnlocked = true;
44         void getBattleAudio().unlock();
45         getBattleAudio().setActive(true);
46     }
47     if (super.handleTouch(ev)) return true;
48     // Content scrolling for guide / campaign text
49     if (this.subpage !== 'guide' && this.subpage !== 'campaign')
50         return false;
```

minigame/src/ui/lobby.ts:376 至 minigame/src/ui/lobby.ts:432

```
1    const p = ev.changed[0];
2    if (!p || p.x < this.contentX()) return false;
3    if (ev.type === 'down') {
4        this.scrollTouchId = p.id;
5        this.scrollStartY = p.y;
6        this.scrollStartScroll = this.scrollY;
7        return true;
8    }
9    if (ev.type === 'move' && this.scrollTouchId === p.id) {
10        this.scrollY = clamp(
11            this.scrollStartScroll - (p.y - this.scrollStartY),
12            0,
13            this.scrollMax,
14        );
15        return true;
16    }
17    if (ev.type === 'up' && this.scrollTouchId === p.id) {
18        this.scrollTouchId = -1;
19        return true;
20    }
21    return false;
22 }
23 // — Drawing —————
24 protected drawSelf(ctx: CanvasRenderingContext2D): void {
25     // Background
26     ctx.fillStyle = COLORS.bg;
27     ctx.fillRect(0, 0, this.screenW, this.screenH);
28     this.drawSidebar(ctx);
29     const cx = this.contentX();
30     ctx.save();
31     ctx.beginPath();
32     ctx.rect(cx, 0, this.contentW(), this.screenH);
33     ctx.clip();
34     ctx.translate(0, -this.scrollY);
35     if (this.subpage === 'home') this.drawHome(ctx);
36     else if (this.subpage === 'guide') this.drawGuide(ctx);
37     else if (this.subpage === 'campaign') this.drawCampaignBrief(ctx);
38     // settings widgets draw themselves; draw labels here
39     if (this.subpage === 'settings') this.drawSettingsLabels(ctx);
40     ctx.restore();
41     // Toast
42     if (this.toastMsg && Date.now() - this.toastTime < 3000) {
43         const alpha = Math.min(1, (3000 - (Date.now() - this.toastTime)) / 500);
44         ctx.globalAlpha = alpha;
45         const w = measureText(ctx, this.toastMsg, 13) + 32;
46         const x = this.screenW / 2 - w / 2;
47         const y = this.screenH - 60;
48         drawPanel(ctx, x, y, w, 32, COLORS.overlay, COLORS.accent);
49         drawTextCentered(ctx, this.toastMsg, this.screenW / 2, y + 16, 13);
50         ctx.globalAlpha = 1;
```

minigame/src/ui/lobby.ts:433 至 minigame/src/ui/lobby.ts:484

```
1    }
2  }
3  private drawSidebar(ctx: CanvasRenderingContext2D): void {
4    drawPanel(ctx, 0, 0, SIDEBAR_W, this.screenH, COLORS.bgPanel, COLORS.border);
5    // Brand
6    drawTextLeft(ctx, '灰线', 12, 20, 20, COLORS.accent, 'bold');
7    drawTextLeft(ctx, 'GREYLINE · v170', 12, 40, 9, COLORS.textDim);
8    // Gold + deck status at bottom
9    const gold = lobbyState.collection.gold;
10   drawTextLeft(ctx, `金币 ${gold}`, 12, this.screenH - 44, 12, COLORS.gold);
11   drawTextLeft(
12     ctx,
13     `当前编队 ${lobbyState.deck.length}/20`,
14     12,
15     this.screenH - 24,
16     12,
17     lobbyState.deck.length === 20 ? COLORS.greenLight : COLORS.redLight,
18   );
19 }
20 private drawHome(ctx: CanvasRenderingContext2D): void {
21   const cx = this.contentX();
22   const cw = this.contentW();
23   if (this.homeBg) {
24     // Cover fit
25     const imgRatio = this.homeBg.width / this.homeBg.height;
26     const boxRatio = cw / this.screenH;
27     let dw = cw;
28     let dh = this.screenH;
29     let dx = cx;
30     let dy = 0;
31     if (imgRatio > boxRatio) {
32       dw = this.screenH * imgRatio;
33       dx = cx - (dw - cw) / 2;
34     } else {
35       dh = cw / imgRatio;
36       dy = -(dh - this.screenH) / 2;
37     }
38     ctx.drawImage(this.homeBg, dx, dy, dw, dh);
39   }
40   // Title
41   drawTextCentered(
42     ctx,
43     '灰线 · 现代战争像素对战',
44     cx + cw / 2,
45     48,
46     22,
47     COLORS.text,
48     'bold',
49   );
50 }
```

minigame/src/ui/lobby.ts:486 至 minigame/src/ui/lobby.ts:537

```

1  private drawGuide(ctx: CanvasRenderingContext2D): void {
2      const cx = this.contentX() + 20;
3      const cw = this.contentW() - 40;
4      let y = 20;
5      drawTextLeft(ctx, '作战手册', cx, y, 20, COLORS.accent, 'bold');
6      y += 36;
7      const sections: [string, string][] = [
8          [
9              '胜利条件',
10             '摧毁敌方指挥部。十分钟后仍未分出胜负，则比较双方基地剩余生命。',
11         ],
12         [
13             '出牌与抽牌',
14             '把卡牌拖出底部扇形区域后松手，即可使用；拖回则取消。长按查看详情。单位从己方基地出发，烟幕、地雷和机降落地使用松手位置。机降直升机会飞抵落点后放下步兵，途中可被防空击落。\\n点击牌堆，消耗 2 点指挥点抽牌，冷却 9 秒。手牌最多 6 张。\\n双方开局 2 点，每 3.6 秒恢复 1 点。后勤可加快回点，扩编可增加上限；难度设置会注明 AI 的额外回点速度。',
15         ],
16         [
17             '查看战场',
18             '左右拖动画面移动视野。手机横屏游玩。',
19         ],
20         [
21             '机步协同',
22             '步兵会自动伴随附近友军坦克，保持后方距离。点击小队可选择伴随、据守、撤退、进攻或警戒；手动进攻会解除伴随。缺少有效反甲或防空保护时，步兵会持续撤离重装火力范围。',
23         ],
24         [
25             '组建编队',
26             '在卡组中选择 20 张牌，每种卡各有数量上限。带上反坦克与防空兵种；AI 使用自己的卡组，双方各抽各的。',
27         ],
28     ];
29     for (const [title, body] of sections) {
30         drawTextLeft(ctx, title, cx, y, 14, COLORS.greenLight, 'bold');
31         y += 22;
32         y = drawWrapped(ctx, body, cx, y, cw, 12, COLORS.text, 'normal', 19) + 12;
33     }
34     this.scrollMax = Math.max(0, y - this.screenH);
35 }
36 private drawCampaignBrief(ctx: CanvasRenderingContext2D): void {
37     const idx = MISSIONS.findIndex((m) => m.id === this.selectedMission);
38     const m = MISSIONS[idx];
39     const cx = this.contentX() + 180;
40     const cw = this.contentW() - 200;
41     let y = 16;
42     drawTextLeft(
43         ctx,
44         '故事战役 · 六章 — 断线之地',
45         cx,
46         y,
47         16,
48         COLORS.accent,
49         'bold',
50     );

```

minigame/src/ui/lobby.ts:538 至 minigame/src/ui/lobby.ts:590

```

1     y += 28;
2     // Landscape image
3     this.loadMissionBg(MAPS[m.mapId].backgroundAsset);
4     const imgH = Math.min(110, this.screenH * 0.3);
5     if (this.missionBg) {
6         ctx.drawImage(this.missionBg, cx, y, cw, imgH);
7     } else {
8         drawPanel(ctx, cx, y, cw, imgH, COLORS.bgPanelLight, COLORS.border);
9     }
10    drawTextLeft(ctx, m.region, cx + 8, y + 14, 11, COLORS.text, 'bold');
11    drawTextLeft(ctx, m.title, cx + 8, y + 30, 15, COLORS.text, 'bold');
12    y += imgH + 12;
13    y = drawWrapped(ctx, m.briefing, cx, y, cw, 12, COLORS.text) + 10;
14    drawTextLeft(ctx, `任务: ${m.goal}`, cx, y, 12, COLORS.greenLight, 'bold');
15    y += 20;
16    y =
17        drawWrapped(ctx, `已知部署: ${m.preparation}`, cx, y, cw, 11, COLORS.textDim) +
18        10;
19    drawTextLeft(
20        ctx,
21        `${DIFFICULTY_LABEL[lobbyState.difficulty]} · ${DIFFICULTY_BONUS[lobbyState.difficulty]}`,
22        cx,
23        y,
24        11,
25        COLORS.textDim,
26    );
27    y += 18;
28    drawTextLeft(
29        ctx,
30        '使用当前20张编队 · 战役进度保存在本机 · 进场先听无线电简报',
31        cx,
32        y,
33        10,
34        COLORS.textMuted,
35    );
36    y += 16;
37    this.scrollMax = Math.max(0, y - this.screenH);
38 }
39 private drawSettingsLabels(ctx: CanvasRenderingContext2D): void {
40     const x = this.contentX() + 24;
41     drawTextLeft(ctx, '对手难度', x, 14, 14, COLORS.accent, 'bold');
42     drawTextLeft(
43         ctx,
44         '双方开局均为 2 点指挥点。资源优势只影响 AI 回点速度。',
45         x,
46         100,
47         11,
48         COLORS.textDim,
49     );
50     drawTextLeft(ctx, '游戏声音', x, 128, 13, COLORS.text);

```

minigame/src/ui/lobby.ts:591 至 minigame/src/ui/shop.ts:24

```
1    const audio = getBattleAudio().settings;
2    drawTextLeft(
3      ctx,
4      `音乐音量 ${Math.round(audio.music * 100)}%`,
5      x + 320,
6      152,
7      12,
8      COLORS.text,
9    );
10   drawTextLeft(
11     ctx,
12     `音效音量 ${Math.round(audio.effects * 100)}%`,
13     x + 320,
14     188,
15     12,
16     COLORS.text,
17   );
18   drawTextLeft(
19     ctx,
20     '自动保存，进入战场后同样生效。',
21     x,
22     224,
23     11,
24     COLORS.textDim,
25   );
26 }
27 }
28 /**
29  * Shop screen: buy card packs, watch the tear animation, reveal draws
30  * (fan for single packs, scrollable grid for ten-packs), watch ads for
31  * gold, and browse the owned-card collection.
32  * Replaces app/shop.tsx.
33  */
34 import {
35   Screen,
36   Button,
37   ScrollList,
38   Widget,
39   type Router,
40 } from './framework';
41 import {
42   COLORS,
43   clamp,
44   drawTextCentered,
45   drawTextLeft,
46   drawTextRight,
47   roundRect,
48 } from './theme';
49 import { drawCardFace, drawCardMini, loadCardImage } from './card-render';
50 import { CardDetailDialog } from './card-detail-dialog';
```

minigame/src/ui/shop.ts:25 至 minigame/src/ui/shop.ts:81

```
1  import { lobbyState } from '../lobby-state';
2  import { CARDS, type CardId } from '@game/engine';
3  import {
4    PACK_COST,
5    TEN_PACK_COST,
6    AD_REWARD,
7    AD_COOLDOWN_MS,
8    RARITY_LABEL,
9    RARITY_RATE,
10   collectionProgress,
11   grantAdReward,
12   openPack,
13   openTenPacks,
14   ownedCount,
15   rarityOf,
16   type PackDraw,
17   type PackResult,
18   type Rarity,
19 } from '@game/collection';
20 import {
21   PACK_CANVAS,
22   PACK_DURATION,
23   packFrameRect,
24   packMotion,
25 } from '@game/pack-animation';
26 import { assetUrl } from '@game/asset-url';
27 const TOP_H = 40;
28 const BOTTOM_H = 100;
29 const LEFT_W = 216;
30 const ROW_H = 50;
31 const RARITY_COLOR: Record<Rarity, string> = {
32   common: '#8b9578',
33   rare: '#c0c8d0',
34   epic: '#e8c25a',
35   legendary: '#b06fd0',
36 };
37 const AD_COOLDOWN_STORAGE = 'greyline-ad-cooldown';
38 // — Owned-card list row —————
39 class OwnedRow extends Widget {
40   id: CardId;
41   count: number;
42   constructor(id: CardId, count: number) {
43     super();
44     this.id = id;
45     this.count = count;
46   }
47   protected drawSelf(ctx: CanvasRenderingContext2D): void {
48     const c = CARDS[this.id];
49     drawCardMini(ctx, this.id, 6, 4, 28);
50     drawTextLeft(ctx, c.name, 44, 13, 12, COLORS.text);
```

minigame/src/ui/shop.ts:82 至 minigame/src/ui/shop.ts:137

```
1    drawTextLeft(
2      ctx,
3      RARITY_LABEL[rarityOf(this.id)],
4      44,
5      30,
6      9,
7      RARITY_COLOR[rarityOf(this.id)],
8    );
9    drawTextRight(ctx, `×${this.count}`, this.w - 8, 26, 12, COLORS.textDim);
10  }
11 }
12 // — Pack tear animation (full screen) —————
13 class TearWidget extends Widget {
14   onDone: (() => void) | null = null;
15   private atlas: HTMLImageElement | null = null;
16   private t = 0;
17   private finished = false;
18   private loadTimer: ReturnType<typeof setTimeout> | null = null;
19   constructor(w: number, h: number) {
20     super();
21     this.w = w;
22     this.h = h;
23     this.loadTimer = setTimeout(() => this.finish(), 5000);
24     loadImage(assetUrl('art/pack-tear-v139.png'))
25       .then((img) => {
26         if (this.loadTimer) clearTimeout(this.loadTimer);
27         this.loadTimer = null;
28         this.atlas = img;
29       })
30       .catch(() => this.finish());
31   }
32   update(dt: number): void {
33     if (!this.atlas || this.finished) return;
34     this.t += dt;
35     if (this.t >= PACK_DURATION) this.finish();
36   }
37   private finish(): void {
38     if (this.finished) return;
39     this.finished = true;
40     if (this.loadTimer) {
41       clearTimeout(this.loadTimer);
42       this.loadTimer = null;
43     }
44     this.onDone?.();
45   }
46   protected drawSelf(ctx: CanvasRenderingContext2D): void {
47     ctx.fillStyle = COLORS.bg;
48     ctx.fillRect(0, 0, this.w, this.h);
49     if (!this.atlas) {
50       drawTextCentered(ctx, '准备拆封...', this.w / 2, this.h / 2, 14, COLORS.textDim);
```

minigame/src/ui/shop.ts:138 至 minigame/src/ui/shop.ts:193

```
1     return;
2   }
3   const motion = packMotion(this.t);
4   const s = Math.min(this.w / PACK_CANVAS.width, this.h / PACK_CANVAS.height) * 0.85;
5   const img = this.atlas;
6   const cell = packFrameRect(img.width, img.height, motion.frame);
7   ctx.save();
8   ctx.imageSmoothingEnabled = false;
9   ctx.globalAlpha = motion.alpha;
10  ctx.translate(this.w / 2, this.h / 2);
11  ctx.scale(s, s);
12  ctx.translate(motion.x - PACK_CANVAS.width / 2, motion.y - PACK_CANVAS.height / 2);
13  ctx.drawImage(
14    img,
15    cell.x,
16    cell.y,
17    cell.width,
18    cell.height,
19    0,
20    0,
21    cell.width,
22    cell.height,
23  );
24  ctx.restore();
25  drawTextCentered(ctx, '点击跳过', this.w / 2, this.h - 24, 11, COLORS.textMuted);
26 }
27 onTouchDown(): boolean {
28   return true;
29 }
30 onTouchUp(): void {
31   this.finish();
32 }
33 }
34 // — Reveal area (fan for 5, scrollable grid for 50) —————
35 interface RevealCallbacks {
36   onFlip: () => void;
37   onFlipAll: () => void;
38   onDetail: (id: CardId) => void;
39 }
40 class RevealAreaWidget extends Widget {
41   private draws: PackDraw[];
42   private flipped: boolean[];
43   private prevOwned: Partial<Record<CardId, number>>;
44   private cb: RevealCallbacks;
45   private backImg: HTMLImageElement | null = null;
46   private ten: boolean;
47   // Grid scroll state
48   private scrolly = 0;
49   private maxScroll = 0;
50   private dragging = false;
```

minigame/src/ui/shop.ts:194 至 minigame/src/ui/shop.ts:248

```
1   private moved = false;
2   private downX = 0;
3   private downY = 0;
4   private startScroll = 0;
5   constructor(
6     w: number,
7     h: number,
8     draws: PackDraw[],
9     flipped: boolean[],
10    prevOwned: Partial<Record<CardId, number>>,
11    cb: RevealCallbacks,
12  ) {
13    super();
14    this.w = w;
15    this.h = h;
16    this.draws = draws;
17    this.flipped = flipped;
18    this.prevOwned = prevOwned;
19    this.cb = cb;
20    this.ten = draws.length > 5;
21    if (this.ten) {
22      const cols = 10;
23      const gap = 6;
24      const cardW = (this.w - 16 - (cols - 1) * gap) / cols;
25      const rows = Math.ceil(draws.length / cols);
26      const contentH = rows * (cardW * 1.5 + gap) + 8;
27      this.maxScroll = Math.max(0, contentH - this.h);
28    }
29    loadImage(assetUrl('art/card-back-v1.webp'))
30      .then((img) => {
31        this.backImg = img;
32      })
33      .catch(() => {});
34  }
35  private isNew(d: PackDraw): boolean {
36    return !d.converted && (this.prevOwned[d.id] ?? 0) === 0;
37  }
38  // — Geometry —
39  private fanCardW(): number {
40    return clamp(96, this.w * 0.14, 130);
41  }
42  /** Returns draw rect + rotation for fan mode, or null if offscreen. */
43  private fanRect(i: number): { x: number; y: number; w: number; h: number; rot: number } {
44    const cw = this.fanCardW();
45    const ch = cw * 1.5;
46    const fanX = clamp(34, this.w * 0.12, 64);
47    const cx = this.w / 2;
48    const cy = this.h * 0.42;
49    const n = this.draws.length;
50    const mid = (n - 1) / 2;
```

minigame/src/ui/shop.ts:249 至 minigame/src/ui/shop.ts:303

```
1    const rot = (i - mid) * 11 * (Math.PI / 180);
2    const spread = fanX + cw * 0.62;
3    const x = cx + (i - mid) * spread - cw / 2;
4    const y = cy + Math.abs(i - mid) * 12 - ch / 2;
5    return { x, y, w: cw, h: ch, rot };
6  }
7  private gridRect(i: number): { x: number; y: number; w: number; h: number } {
8    const cols = 10;
9    const gap = 6;
10   const cw = (this.w - 16 - (cols - 1) * gap) / cols;
11   const ch = cw * 1.5;
12   const col = i % cols;
13   const row = Math.floor(i / cols);
14   return {
15     x: 8 + col * (cw + gap),
16     y: 8 + row * (ch + gap) - this.scrollY,
17     w: cw,
18     h: ch,
19   };
20 }
21 // — Drawing —
22 protected drawSelf(ctx: CanvasRenderingContext2D): void {
23   ctx.fillStyle = COLORS.bg;
24   ctx.fillRect(0, 0, this.w, this.h);
25   ctx.save();
26   ctx.beginPath();
27   ctx.rect(0, 0, this.w, this.h);
28   ctx.clip();
29   if (this.ten) this.drawGrid(ctx);
30   else this.drawFan(ctx);
31   ctx.restore();
32 }
33 private drawFan(ctx: CanvasRenderingContext2D): void {
34   // Draw order: edges first, center last (on top).
35   const n = this.draws.length;
36   const order = [0, n - 1, 1, n - 2, 2].filter((i) => i >= 0 && i < n);
37   const seen = new Set<number>();
38   for (const i of order) {
39     if (seen.has(i)) continue;
40     seen.add(i);
41     this.drawFanCard(ctx, i);
42   }
43   // Any remaining (n>5 safety)
44   for (let i = 0; i < n; i++) {
45     if (!seen.has(i)) this.drawFanCard(ctx, i);
46   }
47 }
48 private drawFanCard(ctx: CanvasRenderingContext2D, i: number): void {
49   const r = this.fanRect(i);
50   const d = this.draws[i];
```

minigame/src/ui/shop.ts:304 至 minigame/src/ui/shop.ts:355

```
1   ctx.save();
2   ctx.translate(r.x + r.w / 2, r.y + r.h / 2);
3   ctx.rotate(r.rot);
4   if (!this.flipped[i]) {
5     this.drawBack(ctx, -r.w / 2, -r.h / 2, r.w, r.h);
6   } else {
7     drawCardFace(ctx, d.id, -r.w / 2, -r.h / 2, r.w);
8     this.drawBadges(ctx, d, -r.w / 2, -r.h / 2, r.w, r.h);
9     // Rarity label under the card
10    const label =
11      RARITY_LABEL[d.rarity] +
12      (this.isNew(d) ? '新卡' : d.converted ? '转化' : '');
13    drawTextCentered(
14      ctx,
15      label,
16      0,
17      r.h / 2 + 10,
18      9,
19      RARITY_COLOR[d.rarity],
20    );
21  }
22  ctx.restore();
23 }
24 private drawGrid(ctx: CanvasRenderingContext2D): void {
25   for (let i = 0; i < this.draws.length; i++) {
26     const r = this.gridRect(i);
27     if (r.y + r.h < 0 || r.y > this.h) continue;
28     const d = this.draws[i];
29     if (!this.flipped[i]) {
30       this.drawBack(ctx, r.x, r.y, r.w, r.h);
31     } else {
32       drawCardFace(ctx, d.id, r.x, r.y, r.w);
33       this.drawBadges(ctx, d, r.x, r.y, r.w, r.h);
34     }
35   }
36 }
37 private drawBack(
38   ctx: CanvasRenderingContext2D,
39   x: number,
40   y: number,
41   w: number,
42   h: number,
43 ): void {
44   if (this.backImg) {
45     ctx.drawImage(this.backImg, x, y, w, h);
46   } else {
47     ctx.fillStyle = '#2a332a';
48     ctx.fillRect(x, y, w, h);
49   }
50   ctx.strokeStyle = COLORS.border;
```

minigame/src/ui/shop.ts:356 至 minigame/src/ui/shop.ts:408

```
1   ctx.lineWidth = 1;
2   ctx.strokeRect(x + 0.5, y + 0.5, w - 1, h - 1);
3 }
4 private drawBadges(
5   ctx: CanvasRenderingContext2D,
6   d: PackDraw,
7   x: number,
8   y: number,
9   w: number,
10  h: number,
11 ): void {
12   if (this.isNew(d)) {
13     ctx.fillStyle = COLORS.red;
14     roundRect(ctx, x + 2, y + 2, 20, 11, 2);
15     ctx.fill();
16     drawTextCentered(ctx, '新', x + 12, y + 8, 9, 'ffffff', 'bold');
17   }
18   if (d.converted) {
19     ctx.fillStyle = COLORS.accentDark;
20     const bw = 34;
21     roundRect(ctx, x + w - bw - 2, y + 2, bw, 11, 2);
22     ctx.fill();
23     drawTextCentered(ctx, `+${d.gold}金`, x + w - bw / 2 - 2, y + 8, 8, 'ffffff');
24   }
25   void h;
26 }
27 // — Hit testing —
28 /** Returns the draw index at a point, or -1. */
29 private hitIndex(px: number, py: number): number {
30   if (this.ten) {
31     for (let i = this.draws.length - 1; i >= 0; i--) {
32       const r = this.gridRect(i);
33       if (px >= r.x && px <= r.x + r.w && py >= r.y && py <= r.y + r.h) return i;
34     }
35     return -1;
36   }
37   // Fan: test in reverse draw order
38   const n = this.draws.length;
39   const order = [2, 3, 1, 4, 0].filter((i) => i >= 0 && i < n);
40   for (const i of order) {
41     const r = this.fanRect(i);
42     // Inverse-rotate the point around the card center
43     const cx = r.x + r.w / 2;
44     const cy = r.y + r.h / 2;
45     const dx = px - cx;
46     const dy = py - cy;
47     const cos = Math.cos(-r.rot);
48     const sin = Math.sin(-r.rot);
49     const lx = dx * cos - dy * sin;
50     const ly = dx * sin + dy * cos;
```

minigame/src/ui/shop.ts:409 至 minigame/src/ui/shop.ts:467

```

1      if (Math.abs(lx) <= r.w / 2 && Math.abs(ly) <= r.h / 2) return i;
2    }
3    return -1;
4  }
5  // — Touch —
6  onTouchDown(x: number, y: number): boolean {
7    this.dragging = true;
8    this.moved = false;
9    this.downX = x;
10   this.downY = y;
11   this.startScroll = this.scrollY;
12   return true;
13 }
14 onTouchMove(x: number, y: number): void {
15   if (!this.dragging) return;
16   const dx = x - this.downX;
17   const dy = y - this.downY;
18   if (Math.abs(dx) > 8 || Math.abs(dy) > 8) this.moved = true;
19   if (this.ten && this.moved) {
20     this.scrollY = clamp(this.startScroll - dy, 0, this.maxScroll);
21   }
22 }
23 onTouchUp(x: number, y: number): void {
24   if (!this.dragging) return;
25   this.dragging = false;
26   if (this.moved) return;
27   const idx = this.hitIndex(x, y);
28   if (idx >= 0) {
29     if (!this.flipped[idx]) {
30       this.flipped[idx] = true;
31       this.cb.onFlip();
32     } else {
33       this.cb.onDetail(this.draws[idx].id);
34     }
35   } else {
36     this.cb.onFlipAll();
37   }
38 }
39 }
40 // — Shop screen —
41 type Phase = 'idle' | 'tearing' | 'fanned';
42 export class ShopScreen extends Screen {
43   private router: Router;
44   private phase: Phase = 'idle';
45   private result: PackResult | null = null;
46   private prevOwned: Partial<Record<CardId, number>> = {};
47   private flipped: boolean[] = [];
48   private message = "";
49   private messageAt = 0;
50   private adReadyAt = 0;

```

minigame/src/ui/shop.ts:469 至 minigame/src/ui/shop.ts:527

```
1   private packImg: HTMLImageElement | null = null;
2   private tearWidget: TearWidget | null = null;
3   private revealWidget: RevealAreaWidget | null = null;
4   private ownedList: ScrollList | null = null;
5   private btnPack: Button | null = null;
6   private btnTen: Button | null = null;
7   private btnAd: Button | null = null;
8   private bottomButtons: Button[] = [];
9   constructor(screenW: number, screenH: number, router: Router) {
10    super(screenW, screenH);
11    this.router = router;
12    try {
13      const saved = Number(localStorage.getItem(AD_COOLDOWN_STORAGE) ?? '0');
14      this.adReadyAt = Number.isFinite(saved) ? saved : 0;
15      if (this.adReadyAt < Date.now()) this.adReadyAt = 0;
16    } catch {
17      this.adReadyAt = 0;
18    }
19    loadCardImage(assetUrl('art/card-pack-v1.webp'))
20      .then((img) => {
21        this.packImg = img;
22      })
23      .catch(() => {});
24  }
25  onEnter(): void {
26    this.buildIdle();
27  }
28  // — Idle layout —
29  private buildIdle(): void {
30    this.phase = 'idle';
31    this.clearChildren();
32    this.result = null;
33    this.bottomButtons = [];
34    // Top bar
35    const back = new Button('返回', 64, 28);
36    back.x = 8;
37    back.y = 6;
38    back.onTap = () => this.router.navigate('lobby');
39    this.addChild(back);
40    // Left column panel content
41    const btnW = 168;
42    const btnX = 8 + (LEFT_W - btnW) / 2;
43    let btnY = TOP_H + 206;
44    this.btnPack = new Button(`开一包 · ${PACK_COST}金`, btnW, 30);
45    this.btnPack.x = btnX;
46    this.btnPack.y = btnY;
47    this.btnPack.onTap = () => this.startReveal(false);
48    this.addChild(this.btnPack);
49    btnY += 34;
50    this.btnTen = new Button(`连开十包 · ${TEN_PACK_COST}金`, btnW, 30);
```

minigame/src/ui/shop.ts:528 至 minigame/src/ui/shop.ts:583

```

1    this.btnTen.x = btnX;
2    this.btnTen.y = btnY;
3    this.btnTen.onTap = () => this.startReveal(true);
4    this.addChild(this.btnTen);
5    btnY += 34;
6    this.btnAd = new Button("", btnW, 30);
7    this.btnAd.x = btnX;
8    this.btnAd.y = btnY;
9    this.btnAd.onTap = () => this.watchAd();
10   this.addChild(this.btnAd);
11   // Right side: owned cards
12   this.ownedList = new ScrollList(
13       this.screenW - 240,
14       this.screenH - 82,
15       ROW_H,
16   );
17   this.ownedList.x = 232;
18   this.ownedList.y = 74;
19   this.ownedList.onItemTap = (index) => {
20       const rows = this.ownedRows();
21       const row = rows[index];
22       if (!row) return;
23       this.showDialog(new CardDetailDialog(row.id, () => this.closeDialog()));
24   };
25   this.addChild(this.ownedList);
26   this.refreshIdle();
27   }
28   private ownedRows(): OwnedRow[] {
29       const col = lobbyState.collection;
30       const ids = (Object.keys(CARDS) as CardId[]).filter(
31           (id) => !CARDS[id].internal && ownedCount(col, id) > 0,
32       );
33       ids.sort((a, b) => {
34           const ca = CARDS[a];
35           const cb = CARDS[b];
36           if (ca.cost !== cb.cost) return ca.cost - cb.cost;
37           return ca.name.localeCompare(cb.name, 'zh-CN');
38       });
39       return ids.map((id) => new OwnedRow(id, ownedCount(col, id)));
40   }
41   private refreshIdle(): void {
42       const col = lobbyState.collection;
43       this.btnPack!.disabled = col.gold < PACK_COST;
44       this.btnTen!.disabled = col.gold < TEN_PACK_COST;
45       this.refreshAdButton();
46       if (this.ownedList) {
47           this.ownedList.setItems(this.ownedRows());
48       }
49   }
50   private refreshAdButton(): void {

```

minigame/src/ui/shop.ts:584 至 minigame/src/ui/shop.ts:637

```
1    if (!this.btnAd) return;
2    const col = lobbyState.collection;
3    const remaining = this.adReadyAt - Date.now();
4    if (remaining > 0) {
5        this.btnAd.label = `广告冷却 ${Math.ceil(remaining / 1000)}s`;
6        this.btnAd.disabled = true;
7    } else {
8        this.btnAd.label = `看广告领 ${AD_REWARD} 金`;
9        this.btnAd.disabled = false;
10   }
11   void col;
12 }
13 // — Pack opening —
14 private startReveal(ten: boolean): void {
15     const col = lobbyState.collection;
16     try {
17         const r = ten ? openTenPacks(col) : openPack(col);
18         this.prevOwned = { ...col.owned };
19         lobbyState.updateCollection(r.state);
20         this.result = r;
21         this.flipped = r.drawn.map(() => false);
22         this.buildTearing();
23     } catch (e) {
24         this.message = e instanceof Error ? e.message : String(e);
25         this.messageAt = Date.now();
26     }
27 }
28 private buildTearing(): void {
29     this.phase = 'tearing';
30     this.clearChildren();
31     this.tearWidget = new TearWidget(this.screenW, this.screenH);
32     this.tearWidget.onDone = () => this.buildFanned();
33     this.addChild(this.tearWidget);
34 }
35 private buildFanned(): void {
36     this.phase = 'fanned';
37     this.clearChildren();
38     this.tearWidget = null;
39     if (!this.result) {
40         this.buildIdle();
41         return;
42     }
43     this.revealWidget = new RevealAreaWidget(
44         this.screenW,
45         this.screenH - BOTTOM_H,
46         this.result.drawn,
47         this.flipped,
48         this.prevOwned,
49         {
50             onFlip: () => this.refreshBottomBar(),
```

minigame/src/ui/shop.ts:638 至 minigame/src/ui/shop.ts:691

```
1      onFlipAll: () => {
2          for (let i = 0; i < this.flipped.length; i++) this.flipped[i] = true;
3          this.refreshBottomBar();
4      },
5      onDetail: (id) => {
6          this.showDialog(new CardDetailDialog(id, () => this.closeDialog()));
7      },
8  },
9  );
10  this.addChild(this.revealWidget);
11  this.refreshBottomBar();
12  }
13  private get allFlipped(): boolean {
14      return this.flipped.every((f) => f);
15  }
16  private refreshBottomBar(): void {
17      for (const b of this.bottomButtons) this.removeChild(b);
18      this.bottomButtons = [];
19      if (this.phase !== 'fanned' || !this.result) return;
20      const y = this.screenH - 62;
21      const h = 30;
22      let x = 8;
23      const add = (label: string, w: number, onTap: () => void, disabled = false) => {
24          const b = new Button(label, w, h);
25          b.x = x;
26          b.y = y;
27          b.onTap = onTap;
28          b.disabled = disabled;
29          this.addChild(b);
30          this.bottomButtons.push(b);
31          x += w + 6;
32      };
33      if (!this.allFlipped) {
34          add('全部翻开', 84, () => {
35              for (let i = 0; i < this.flipped.length; i++) this.flipped[i] = true;
36              this.refreshBottomBar();
37          });
38      }
39      add('收下', 72, () => this.buildIdle(), !this.allFlipped);
40      add(
41          '再开一包',
42          92,
43          () => this.startReveal(false),
44          lobbyState.collection.gold < PACK_COST,
45      );
46      add(
47          '再开十包',
48          92,
49          () => this.startReveal(true),
50          lobbyState.collection.gold < TEN_PACK_COST,
```

minigame/src/ui/shop.ts:692 至 minigame/src/ui/shop.ts:749

```

1   );
2   }
3   private summaryText(): string {
4       if (!this.result) return "";
5       const newCount = this.result.drawn.filter(
6           (d) => !d.converted && (this.prevOwned[d.id] ?? 0) === 0,
7       ).length;
8       let parts = [`新卡 ${newCount} 张`];
9       if (this.result.goldGained > 0) {
10          parts.push(`溢出转化 +${this.result.goldGained} 金币`);
11      }
12      if (this.result.pity) {
13          parts.push('保底王牌已触发');
14      }
15      return parts.join(' · ');
16  }
17  // — Ads —
18  private watchAd(): void {
19      if (this.adReadyAt > Date.now()) return;
20      lobbyState.updateCollection(grantAdReward(lobbyState.collection));
21      this.adReadyAt = Date.now() + AD_COOLDOWN_MS;
22      try {
23          localStorage.setItem(AD_COOLDOWN_STORAGE, String(this.adReadyAt));
24      } catch {
25          /* storage may be unavailable */
26      }
27      this.message = `广告奖励 +${AD_REWARD} 金币`;
28      this.messageAt = Date.now();
29      this.refreshIdle();
30  }
31  // — Frame —
32  update(dt: number): void {
33      if (this.phase === 'tearing' && this.tearWidget) {
34          this.tearWidget.update(dt);
35      } else if (this.phase === 'idle') {
36          this.refreshAdButton();
37      }
38  }
39  protected drawSelf(ctx: CanvasRenderingContext2D): void {
40      ctx.fillStyle = COLORS.bg;
41      ctx.fillRect(0, 0, this.screenW, this.screenH);
42      if (this.phase === 'tearing') return;
43      if (this.phase === 'idle') {
44          this.drawIdle(ctx);
45      } else if (this.phase === 'fanned') {
46          drawTextCentered(
47              ctx,
48              this.summaryText(),
49              this.screenW / 2,
50              this.screenH - 86,

```

minigame/src/ui/shop.ts:750 至 minigame/src/ui/shop.ts:802

```
1      12,
2      COLORS.accent,
3      'bold',
4  );
5  if (!this.allFlipped) {
6      drawTextCentered(
7          ctx,
8          '点击卡片翻一张 · 点击桌面全翻',
9          this.screenW / 2,
10         this.screenH - 14,
11         10,
12         COLORS.textMuted,
13     );
14 }
15 }
16 }
17 private drawIdle(ctx: CanvasRenderingContext2D): void {
18     // Top bar title + gold
19     drawTextCentered(ctx, '军需商店', this.screenW / 2, 20, 15, COLORS.text, 'bold');
20     drawTextRight(
21         ctx,
22         `金币 ${lobbyState.collection.gold}`,
23         this.screenW - 12,
24         20,
25         13,
26         COLORS.accent,
27         'bold',
28     );
29     // Left panel background
30     ctx.fillStyle = COLORS.bgPanel;
31     roundRect(ctx, 8, TOP_H + 4, LEFT_W, this.screenH - TOP_H - 12, 6);
32     ctx.fill();
33     ctx.strokeStyle = COLORS.border;
34     ctx.lineWidth = 1;
35     ctx.stroke();
36     // Pack image
37     const packW = 100;
38     const packH = clamp(this.screenH - 220, 90, 150);
39     const packX = 8 + (LEFT_W - packW) / 2;
40     const packY = TOP_H + 24;
41     if (this.packImg) {
42         ctx.drawImage(this.packImg, packX, packY, packW, packH);
43     } else {
44         ctx.fillStyle = COLORS.bgPanelLight;
45         roundRect(ctx, packX, packY, packW, packH, 4);
46         ctx.fill();
47     }
48     drawTextCentered(
49         ctx,
50         '前线卡包',
```

minigame/src/ui/shop.ts:803 至 minigame/src/ui/theme.ts:3

```
1      8 + LEFT_W / 2,
2      packY + packH + 14,
3      12,
4      COLORS.text,
5  );
6  // Rates
7  const rateY = TOP_H + 206 + 34 * 3 + 40;
8  drawTextCentered(
9      ctx,
10     `常规 ${((RARITY_RATE.common * 100).toFixed(0))}% · 精锐 ${((RARITY_RATE.rare * 100).toFixed(0))}%`,
11     8 + LEFT_W / 2,
12     rateY,
13     9,
14     COLORS.textMuted,
15 );
16 drawTextCentered(
17     ctx,
18     `王牌 ${((RARITY_RATE.epic * 100).toFixed(0))}% · 传奇 ${((RARITY_RATE.legendary * 100).toFixed(0))}%`,
19     8 + LEFT_W / 2,
20     rateY + 14,
21     9,
22     COLORS.textMuted,
23 );
24 // Message
25 if (this.message && Date.now() - this.messageAt < 4000) {
26     drawTextCentered(
27         ctx,
28         this.message,
29         8 + LEFT_W / 2,
30         rateY + 40,
31         11,
32         COLORS.redLight,
33     );
34 }
35 // Right side header
36 const col = lobbyState.collection;
37 const prog = collectionProgress(col);
38 drawTextLeft(
39     ctx,
40     `已收集 ${prog.species}/${prog.total} 种 · 共 ${prog.copies} 张 · 已开 ${col.packsOpened} 包`,
41     232,
42     52,
43     11,
44     COLORS.textDim,
45 );
46 drawTextLeft(ctx, '我的卡牌', 232, 66, 12, COLORS.text, 'bold');
47 }
48 }
49 /** Theme constants shared by all Canvas UI pages. */
50 export const COLORS = {
```

minigame/src/ui/theme.ts:4 至 minigame/src/ui/theme.ts:57

```
1   bg: '#1a1f1a',
2   bgPanel: '#232b23',
3   bgPanelLight: '#2d3a2d',
4   bgCard: '#2a332a',
5   border: '#4a5a4a',
6   borderLight: '#6a7a6a',
7   text: '#e8e0d0',
8   textDim: '#9a9a8a',
9   textMuted: '#6a6a5a',
10  accent: '#d4a843',
11  accentDark: '#8a6a20',
12  green: '#5a8a4a',
13  greenLight: '#7aaa5a',
14  red: '#b5453a',
15  redLight: '#d4655a',
16  blue: '#4a7ab5',
17  blueLight: '#6a9ad4',
18  gold: '#d4a843',
19  hp: '#c44a3a',
20  hpBg: '#3a2020',
21  energy: '#d4a843',
22  energyBg: '#3a3020',
23  overlay: 'rgba(10, 12, 10, 0.85)',
24  scrim: 'rgba(10, 12, 10, 0.6)',
25 } as const;
26 export const FONT = {
27   pixel: 'fusion-pixel',
28   fallback: 'monospace',
29 } as const;
30 export const LAYOUT = {
31   /** Standard margin from screen edge. */
32   margin: 8,
33   /** Standard widget gap. */
34   gap: 6,
35   /** Button height. */
36   btnH: 36,
37   /** Small button height. */
38   btnSmH: 28,
39   /** Panel corner radius. */
40   radius: 4,
41   /** Top bar height. */
42   topBarH: 40,
43   /** Bottom bar height. */
44   bottomBarH: 120,
45 } as const;
46 export function font(size: number, weight: 'normal' | 'bold' = 'normal'): string {
47   return `${weight === 'bold' ? 'bold' : ''}${size}px "${FONT.pixel}", ${FONT.fallback}`;
48 }
49 /** Measure text width using the pixel font. */
50 export function measureText(
```

minigame/src/ui/theme.ts:58 至 minigame/src/ui/theme.ts:110

```
1   ctx: CanvasRenderingContext2D,  
2   text: string,  
3   size: number,  
4   weight: 'normal' | 'bold' = 'normal',  
5 ): number {  
6   ctx.font = font(size, weight);  
7   return ctx.measureText(text).width;  
8 }  
9 /** Draw text centered horizontally at (cx, y). */  
10 export function drawTextCentered(  
11   ctx: CanvasRenderingContext2D,  
12   text: string,  
13   cx: number,  
14   y: number,  
15   size: number,  
16   color: string = COLORS.text,  
17   weight: 'normal' | 'bold' = 'normal',  
18 ): void {  
19   ctx.font = font(size, weight);  
20   ctx.fillStyle = color;  
21   ctx.textAlign = 'center';  
22   ctx.textBaseline = 'middle';  
23   ctx.fillText(text, cx, y);  
24 }  
25 /** Draw text left-aligned at (x, y). */  
26 export function drawTextLeft(  
27   ctx: CanvasRenderingContext2D,  
28   text: string,  
29   x: number,  
30   y: number,  
31   size: number,  
32   color: string = COLORS.text,  
33   weight: 'normal' | 'bold' = 'normal',  
34 ): void {  
35   ctx.font = font(size, weight);  
36   ctx.fillStyle = color;  
37   ctx.textAlign = 'left';  
38   ctx.textBaseline = 'middle';  
39   ctx.fillText(text, x, y);  
40 }  
41 /** Draw text right-aligned at (x, y). */  
42 export function drawTextRight(  
43   ctx: CanvasRenderingContext2D,  
44   text: string,  
45   x: number,  
46   y: number,  
47   size: number,  
48   color: string = COLORS.text,  
49   weight: 'normal' | 'bold' = 'normal',  
50 ): void {
```

minigame/src/ui/theme.ts:111 至 minigame/src/ui/theme.ts:163

```
1   ctx.font = font(size, weight);
2   ctx.fillStyle = color;
3   ctx.textAlign = 'right';
4   ctx.textBaseline = 'middle';
5   ctx.fillText(text, x, y);
6 }
7 /** Draw a rounded rectangle path. */
8 export function roundRect(
9   ctx: CanvasRenderingContext2D,
10  x: number,
11  y: number,
12  w: number,
13  h: number,
14  r: number,
15 ): void {
16   const rr = Math.min(r, w / 2, h / 2);
17   ctx.beginPath();
18   ctx.moveTo(x + rr, y);
19   ctx.lineTo(x + w - rr, y);
20   ctx.arcTo(x + w, y, x + w, y + rr, rr);
21   ctx.lineTo(x + w, y + h - rr);
22   ctx.arcTo(x + w, y + h, x + w - rr, y + h, rr);
23   ctx.lineTo(x + rr, y + h);
24   ctx.arcTo(x, y + h, x, y + h - rr, rr);
25   ctx.lineTo(x, y + rr);
26   ctx.arcTo(x, y, x + rr, y, rr);
27   ctx.closePath();
28 }
29 /** Draw a filled panel with border. */
30 export function drawPanel(
31   ctx: CanvasRenderingContext2D,
32   x: number,
33   y: number,
34   w: number,
35   h: number,
36   fill: string = COLORS.bgPanel,
37   border: string = COLORS.border,
38   radius: number = LAYOUT.radius,
39 ): void {
40   roundRect(ctx, x, y, w, h, radius);
41   ctx.fillStyle = fill;
42   ctx.fill();
43   ctx.strokeStyle = border;
44   ctx.lineWidth = 1;
45   ctx.stroke();
46 }
47 /** Draw a progress bar. */
48 export function drawBar(
49   ctx: CanvasRenderingContext2D,
50   x: number,
```

minigame/src/ui/theme.ts:164 至 minigame/src/ui/theme.ts:216

```

1   y: number,
2   w: number,
3   h: number,
4   pct: number,
5   fillColor: string,
6   bgColor: string = COLORS.hpBg,
7 ): void {
8   ctx.fillStyle = bgColor;
9   ctx.fillRect(x, y, w, h);
10  const fw = Math.max(0, Math.min(1, pct)) * w;
11  if (fw > 0) {
12    ctx.fillStyle = fillColor;
13    ctx.fillRect(x, y, fw, h);
14  }
15  ctx.strokeStyle = COLORS.border;
16  ctx.lineWidth = 1;
17  ctx.strokeRect(x + 0.5, y + 0.5, w - 1, h - 1);
18 }
19 /** Clamp a value. */
20 export function clamp(v: number, lo: number, hi: number): number {
21   return Math.max(lo, Math.min(hi, v));
22 }
23 /** Linear interpolate. */
24 export function lerp(a: number, b: number, t: number): number {
25   return a + (b - a) * t;
26 }
27 /**
28  * Wrap text to fit a width. Returns lines. Handles CJK character-by-character
29  * and Latin word-by-word wrapping.
30  */
31 export function wrapText(
32   ctx: CanvasRenderingContext2D,
33   text: string,
34   maxWidth: number,
35   size: number,
36   weight: 'normal' | 'bold' = 'normal',
37 ): string[] {
38   ctx.font = font(size, weight);
39   const lines: string[] = [];
40   const paragraphs = text.split('\n');
41   for (const para of paragraphs) {
42     let line = "";
43     let i = 0;
44     while (i < para.length) {
45       const ch = para[i];
46       // Latin word: grab until whitespace
47       if (/[-~]/.test(ch) && ch !== ' ') {
48         let j = i;
49         while (j < para.length && /^[^\s—-□-◻◻◻]/.test(para[j])) j++;
50         const word = para.slice(i, j);

```

minigame/src/ui/theme.ts:217 至 minigame/src/ui/theme.ts:267

```
1      const test = line ? line + word : word;
2      if (ctx.measureText(test).width > maxWidth && line) {
3          lines.push(line);
4          line = word;
5      } else {
6          line = test;
7      }
8      i = j;
9  } else {
10     const test = line + ch;
11     if (ctx.measureText(test).width > maxWidth && line) {
12         lines.push(line);
13         line = ch === ' ' ? ' ' : ch;
14     } else {
15         line = test;
16     }
17     i++;
18 }
19 }
20 lines.push(line);
21 }
22 return lines;
23 }
24 /**
25  * Draw wrapped text starting at (x, y). Returns the y position after the
26  * last line.
27  */
28 export function drawWrapped(
29     ctx: CanvasRenderingContext2D,
30     text: string,
31     x: number,
32     y: number,
33     maxWidth: number,
34     size: number,
35     color: string = COLORS.text,
36     weight: 'normal' | 'bold' = 'normal',
37     lineHeight: number = size * 1.5,
38 ): number {
39     const lines = wrapText(ctx, text, maxWidth, size, weight);
40     ctx.font = font(size, weight);
41     ctx.fillStyle = color;
42     ctx.textAlign = 'left';
43     ctx.textBaseline = 'top';
44     let cy = y;
45     for (const line of lines) {
46         ctx.fillText(line, x, cy);
47         cy += lineHeight;
48     }
49     return cy;
50 }
```